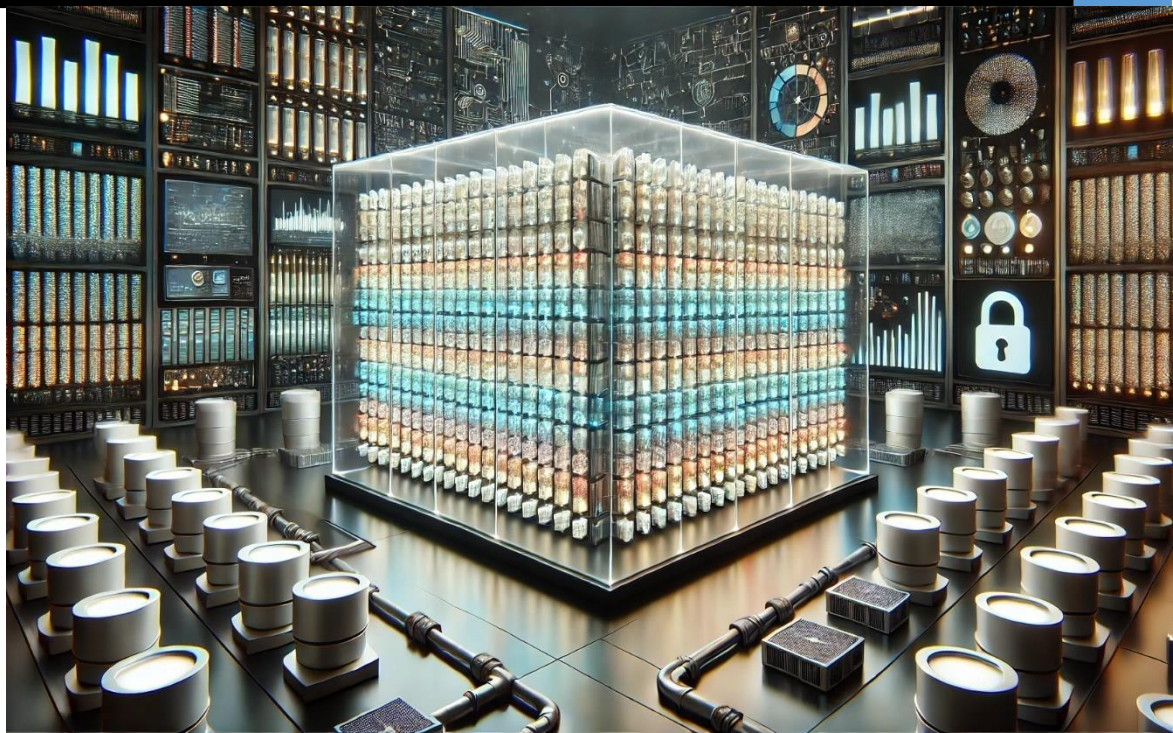


UD 2. Diseño y construcción de soluciones en sistemas de almacenamiento de datos



Curso de Especialización en IA y BD

5075. Big Data aplicado

Prof. D. Ricardo Mariscal Quintero

Índice General

Índice de Figuras	5
Información previa	7
Actividades previas.....	8
1. Introducción a los sistemas de almacenamiento de datos.....	10
1.1. Definición y objetivos de los sistemas de almacenamiento de datos.....	10
1.1.1. Diferencia entre almacenamiento operativo (OLTP) y analítico (OLAP)	11
1.2. Almacenes de datos vs. Lagos de datos	12
1.2.1. Características, diferencias y aplicaciones	12
1.2.2. Cuándo elegir un Almacén de datos y cuándo un Lago de datos.....	14
1.3. Evolución del almacenamiento de datos	16
1.3.1. De las bases de datos relacionales tradicionales (SQL) a los enfoques modernos (NoSQL, Data Lakes)	16
1.3.2. Creciente demanda por manejo de datos no estructurados	17
2. Diseño de arquitecturas de almacenamiento	18
2.1. Modelos de diseño en Almacenes de datos: Enfoques Kimball e Inmon.....	18
2.1.1. Introducción a los dos modelos de diseño de Almacenes de datos más utilizados	18
2.1.2. Enfoque de Ralph Kimball.....	20
2.1.3. Enfoque de Bill Inmon	23
2.2. OLTP (Online Transaction Processing) vs. OLAP (Online Analytical Processing)	26

2.2.1. Diferencias y similitudes en cuanto a diseño, enfoque y uso	26
2.2.2. Requisitos de rendimiento y cómo afectan el diseño arquitectónico	27
2.3. ETL (Extracción, Transformación y Carga) vs. ELT (Extracción, Carga y Transformación).....	28
2.3.1. Importancia en los Almacenes de datos	30
3. Soluciones de almacenamiento en la nube	31
3.1. Ventajas del almacenamiento en la nube	31
3.1.1. Escalabilidad, flexibilidad y modelos de pago por uso	31
3.1.2. Reducción de costos operativos	32
3.2. Data Warehouses en la nube	32
3.2.1. Soluciones como Amazon Redshift, Google BigQuery y Snowflake.....	33
3.2.2. Casos de uso y consideraciones al migrar a la nube	34
3.3. Lagos de datos en la nube	35
3.3.1. Implementación de soluciones como Azure Data Lake y Amazon S3	35
3.3.2. Ventajas para almacenamiento de grandes volúmenes de datos no estructurados.....	36
3.4. Multinube y soluciones híbridas	36
3.4.1. Uso de múltiples proveedores de nube y arquitecturas híbridas	36
3.4.2. Gestión y optimización del rendimiento y costos	37
4. Construcción de almacenes de datos	38
4.1. Esquemas en estrella vs. Cubos OLAP	39
4.2. Tablas de hechos para medir	42

4.3. Tablas de dimensiones para describir el contexto	45
4.4. Hechos y dimensiones unidos en un esquema de estrella.....	46
4.5. Conceptos generales asociados a los hechos	47
4.5.1 Datos aditivos, semi-aditivos y no aditivos.....	47
4.5.2 Valores nulos.....	50
4.5.3 Year-To-Date (YTD).....	53
4.5.4 Tipos de tablas de hechos.....	55
4.5.5 Tablas de hechos sin hechos.....	57
4.5.6 Claves subrogadas	59
4.6 Conceptos generales asociados a las dimensiones.....	60
4.6.1 Dimensión Fecha.....	64
4.6.2 Nulos en dimensiones.....	66
4.6.3 Jerarquías en dimensiones	68
4.6.4 Dimensiones conformadas	71
4.6.5 Dimensiones degeneradas	72
4.6.6 Dimensiones basura.....	74
4.6.7 Dimensiones role-playing	75
4.6.8 Dimensiones de cambio lento	77
4.7 Desarrollando el diseño dimensional	84
5. Implementación de bases de datos NoSQL	88
5.1 Introducción a NoSQL.....	88

5.1.1. Diferencias entre bases de datos relacionales y NoSQL	88
5.1.2. Modelo ACID y teorema CAP	89
5.1.3. Cuándo utilizar bases de datos NoSQL en lugar de SQL	92
5.2 Tipos de bases de datos NoSQL	92
5.2.1. Bases de datos documentales	92
5.2.2. Bases de datos clave-valor	93
5.2.3. Bases de datos columnar	93
5.2.4. Bases de datos de grafos.....	93
5.3 Modelado en NoSQL	94
5.3.1. Enfoques de modelado de datos en NoSQL.....	94
5.3.2. Ejemplos prácticos	94
5.4 Integración con almacenes de datos y lagos de datos	95
5.5 Gestión del rendimiento y escalabilidad	95
5.5.1. Diseño para grandes volúmenes de datos.....	95
Bibliografía.....	97

Índice de Figuras

Figura 1. Sistemas de almacenamiento de datos	10
Figura 2. OLTP vs. OLAP.....	11
Figura 3. Almacén de datos	13
Figura 4. Lago de datos	14
Figura 5. Almacén de datos - Datos estructurados	15
Figura 6. Lago de datos – Datos semi-estructurados y no estructurados	15
Figura 7. SQL vs. NoSQL	16
Figura 8. Enfoques Ralph Kimball vs. Bill Inmon	19
Figura 9. Enfoque Ralph Kimball	21
Figura 10. Esquema estrella.....	22
Figura 11. Esquema copo de nieve	22
Figura 12. Enfoque Bill Inmon.....	24
Figura 13. Esquema en estrella vs. Cubos OLAP.....	39
Figura 14. Modelo dimensional.....	40
Figura 15. Cubo OLAP.....	41
Figura 16. Modelo dimensional.....	46
Figura 17. Jerarquías balanceadas	69
Figura 18. Jerarquías no balanceadas	69
Figura 19. Jerarquía con niveles opcionales	70
Figura 20. Dimensiones conformadas	71

Figura 21. SCD tipo 1.....	78
Figura 22. SCD tipo 2.....	79
Figura 23. SCD tipo 3.....	80
Figura 24. SCD tipo 4.....	81
Figura 25. SCD tipo 5.....	82
Figura 26. SCD tipo 6.....	83
Figura 27. SCD tipo 7.....	84
Figura 28. Teorema CAP.....	89

Información previa

Esta unidad didáctica trabaja la adquisición de la competencia “Gestiona soluciones a problemas propuestos, utilizando sistemas de almacenamiento y herramientas asociadas al centro de datos. (RA 1)”.

Los objetivos a cubrir, durante todo el desarrollo de la presente unidad, son:

- Caracterizar el proceso de diseño y construcción de soluciones en sistemas de almacenamiento de datos. (CE a)

Los contenidos que se van a desarrollar serán:

- Introducción a los Sistemas de Almacenamiento de Datos
- Diseño de Arquitecturas de Almacenamiento
- Soluciones de Almacenamiento en la Nube
- Construcción de Almacenes de Datos
- Implementación de Bases de Datos NoSQL

Actividades previas

Sin haber revisado aún los contenidos de la unidad, investiga para responder a las siguientes preguntas:

1. Explica la diferencia entre un sistema de almacenamiento operativo (OLTP) y un sistema de almacenamiento analítico (OLAP).
2. Investiga qué es un lago de datos y en qué se diferencia de un almacén de datos.
3. ¿Cuáles son las ventajas de usar un almacén de datos para análisis frente a otras opciones como un lago de datos?
4. Describe el proceso de evolución del almacenamiento de datos desde las bases de datos relacionales tradicionales hasta las modernas bases de datos NoSQL.
5. ¿Qué desafíos crees que se enfrentan al manejar grandes volúmenes de datos no estructurados en un lago de datos?
6. Explica en qué consiste el enfoque de diseño de almacenes de datos de Ralph Kimball y su aplicación práctica.
7. Analiza las diferencias clave entre los enfoques de diseño de almacenes de datos de Kimball e Inmon.
8. Explica el concepto de un esquema en estrella y su relevancia en el diseño de almacenes de datos.
9. Investiga las principales diferencias entre los procesos de ETL y ELT. ¿Cuándo es más recomendable usar uno u otro?
10. ¿Cuáles son las ventajas del uso del almacenamiento en la nube frente a soluciones locales (on-premise)?
11. Explora las características de soluciones de almacenamiento en la nube como Amazon Redshift, Google BigQuery y Snowflake.
12. Describe el impacto que tiene el formato de los datos en el rendimiento y la escalabilidad de un sistema de almacenamiento en Big Data.

13. Investiga qué son las bases de datos NoSQL y en qué situaciones serían más apropiadas que una base de datos SQL.
14. ¿Qué es la escalabilidad en el contexto de sistemas de almacenamiento de datos? Explica las diferencias entre escalabilidad horizontal y vertical.
15. Analiza cómo el uso de arquitecturas de almacenamiento híbridas (nube y local) puede beneficiar a una organización.

1. Introducción a los sistemas de almacenamiento de datos

En la era digital actual, las organizaciones se enfrentan al desafío constante de gestionar volúmenes cada vez mayores de información. Los sistemas de almacenamiento de datos han surgido como soluciones esenciales para recopilar, organizar y administrar estos datos de manera eficiente. No se trata solo de guardar información, sino de garantizar que esté disponible y sea útil para quienes la necesitan, cuando la necesitan. Estos sistemas permiten a las empresas acceder rápidamente a datos críticos, procesarlos y analizarlos para tomar decisiones informadas que impulsan estrategias y operaciones.

1.1. Definición y objetivos de los sistemas de almacenamiento de datos

Un sistema de almacenamiento de datos es una combinación integrada de tecnologías, herramientas y prácticas diseñadas para manejar grandes cantidades de información. Su objetivo principal es proporcionar una manera eficiente y segura de almacenar datos, ya sean estructurados como bases de datos tradicionales o no estructurados como correos electrónicos, documentos y multimedia. Al hacerlo, facilitan el acceso rápido y fiable a la información, tanto para operaciones diarias como para análisis profundos.

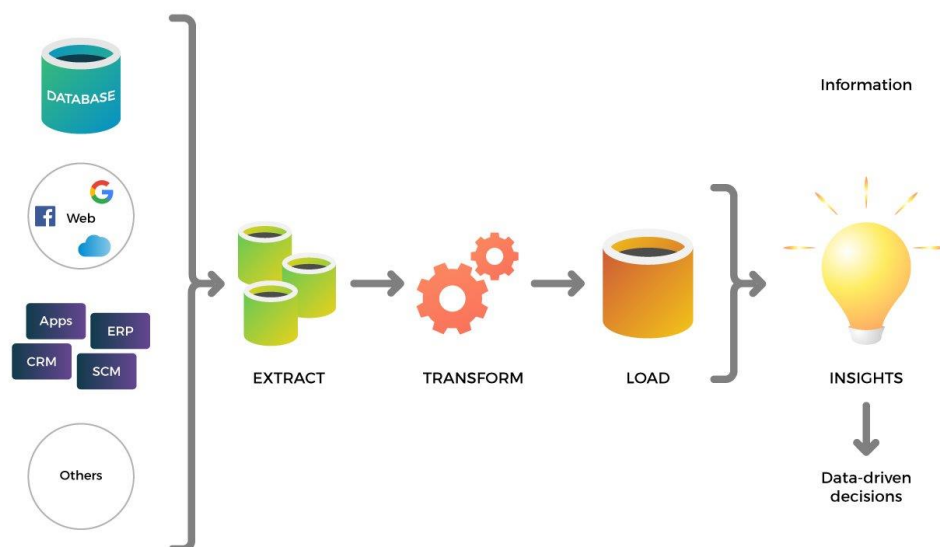


Figura 1. Sistemas de almacenamiento de datos

Fuente: blog.bismart.com

Estos sistemas también juegan un papel crucial en la consolidación y organización de datos provenientes de diversas fuentes. Al unificar esta información, las organizaciones pueden obtener una visión integrada y coherente de sus operaciones, lo que es fundamental para la planificación y la toma de decisiones estratégicas. Además, permiten realizar análisis detallados y generar informes que ayudan a identificar tendencias, oportunidades y áreas de mejora.

En esencia, el objetivo de un sistema de almacenamiento de datos es asegurar que la información esté disponible y sea accesible cuando y donde se necesite. Debe ser flexible y escalable, capaz de adaptarse al crecimiento y los cambios en los datos a lo largo del tiempo, y de soportar las necesidades cambiantes de la organización.

1.1.1. Diferencia entre almacenamiento operativo (OLTP) y analítico (OLAP)

Dentro del ámbito de los sistemas de almacenamiento de datos, es importante distinguir entre el procesamiento de transacciones en línea (OLTP) y el procesamiento analítico en línea (OLAP), ya que cada uno sirve a propósitos diferentes y tiene características específicas.

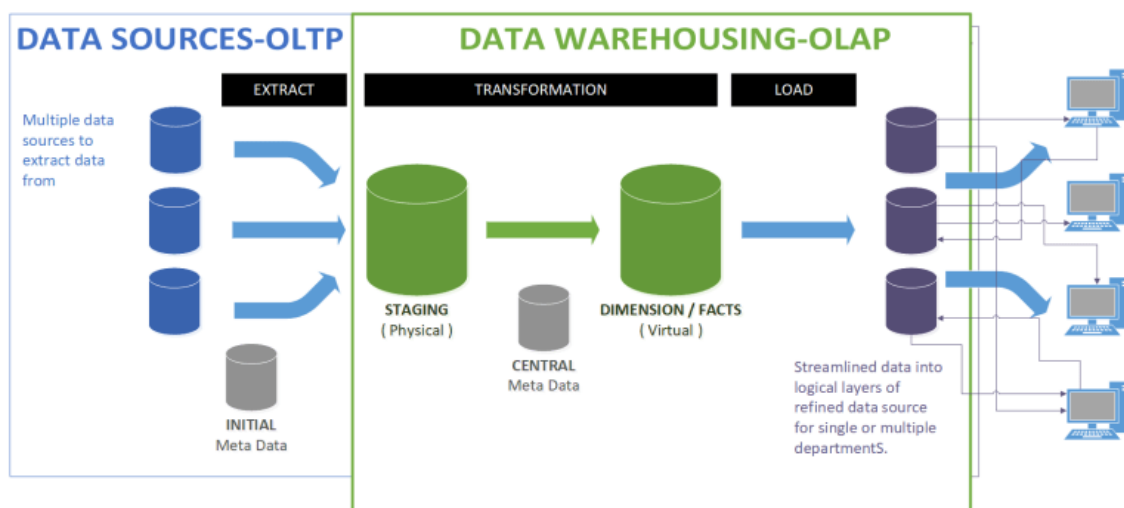


Figura 2. OLTP vs. OLAP

Fuente: carlospesquera.com

OLTP (Online Transaction Processing) se refiere a sistemas diseñados para gestionar las operaciones transaccionales diarias de una organización en tiempo real. Estos sistemas manejan un gran número de transacciones simultáneamente, garantizando la integridad y consistencia de los datos con cada operación. Las bases de datos OLTP están altamente normalizadas para evitar la duplicación de datos y mantener su consistencia. Por ejemplo, en un sistema bancario, cada vez que un cliente realiza una transferencia o un pago, el sistema OLTP actualiza instantáneamente los saldos y registra la transacción, asegurando que la información esté siempre actualizada y sea fiable.

Por otro lado, **OLAP (Online Analytical Processing)** se enfoca en el análisis y consulta de grandes volúmenes de datos históricos. Estos sistemas permiten a las organizaciones realizar análisis complejos, identificar patrones y tendencias, y apoyar la toma de decisiones estratégicas. Las bases de datos OLAP suelen estar desnormalizadas para mejorar el rendimiento de las consultas y facilitar el acceso rápido a los datos. Un ejemplo sería un sistema que analiza los datos de ventas de una empresa a lo largo de varios años para identificar tendencias de consumo, evaluar el rendimiento de productos y ayudar a planificar estrategias de mercado.

En resumen, mientras que OLTP se centra en el procesamiento eficiente de transacciones diarias, OLAP se dedica al análisis profundo de datos para apoyar decisiones a largo plazo.

1.2. Almacenes de datos vs. Lagos de datos

A medida que las organizaciones buscan aprovechar al máximo sus datos, han surgido dos enfoques principales para su almacenamiento y gestión: los almacenes de datos y los lagos de datos. Aunque ambos sirven para almacenar información, difieren significativamente en su estructura, metodología y casos de uso.

1.2.1. Características, diferencias y aplicaciones

Los **almacenes de datos** son bases de datos altamente estructuradas orientadas al análisis de datos históricos consolidados de diferentes fuentes dentro de una organización. Están

diseñados para soportar consultas complejas y análisis que ayudan en la toma de decisiones estratégicas. Antes de ingresar al almacén, los datos pasan por procesos de extracción, transformación y carga (ETL), donde se limpian y organizan para garantizar su calidad y consistencia. Este enfoque es ideal para informes de gestión y análisis empresariales que requieren datos precisos y estandarizados, como en los departamentos de finanzas o marketing.

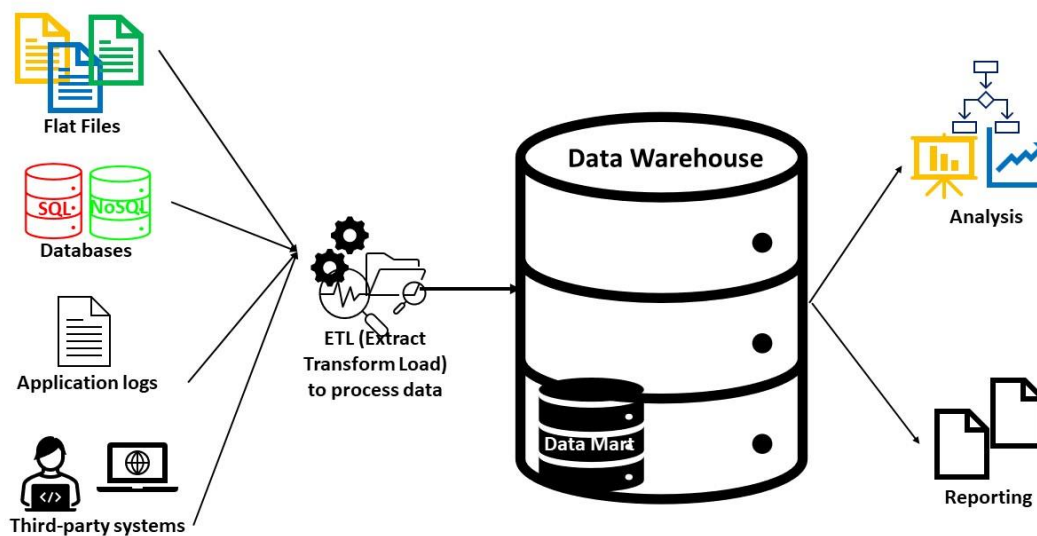


Figura 3. Almacén de datos

Fuente: geekflare.com

En contraste, los **lagos de datos** son repositorios centralizados que permiten almacenar grandes cantidades de datos en su forma original y sin procesar. Soportan una variedad de formatos, incluyendo datos estructurados, semi-estructurados y no estructurados. Los datos se almacenan tal cual llegan, y las transformaciones necesarias se aplican durante el análisis (conocido como "schema-on-read"). Este enfoque es ideal para escenarios que requieren análisis avanzados y flexibles, como proyectos de Big Data, inteligencia artificial y aprendizaje automático. Los lagos de datos son especialmente útiles para manejar datos generados por dispositivos IoT, logs de servidores o información de redes sociales.

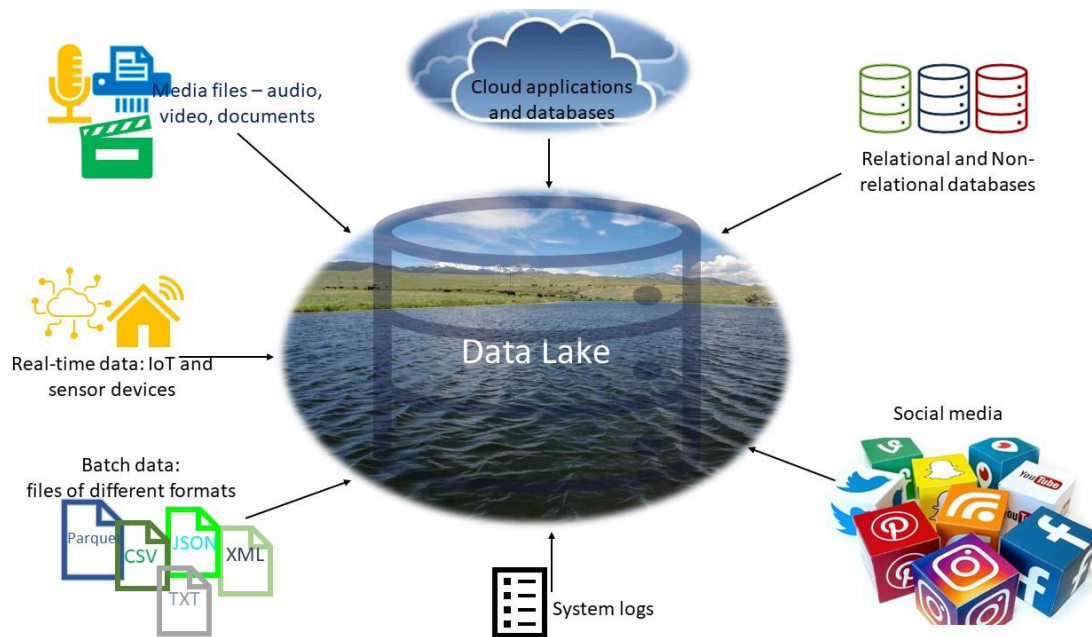


Figura 4. Lago de datos

Fuente: geekflare.com

La elección entre un almacén de datos y un lago de datos depende de las necesidades específicas de la organización. Si se requiere un análisis estructurado y consistente para informes y decisiones estratégicas, un almacén de datos es la opción adecuada. Si, por el contrario, la organización necesita almacenar y analizar grandes volúmenes de datos diversos y no estructurados, un lago de datos ofrece la flexibilidad y escalabilidad necesarias.

1.2.2. Cuándo elegir un Almacén de datos y cuándo un Lago de datos

Optar por un almacén de datos es recomendable cuando la organización necesita informes precisos y coherentes basados en datos estructurados y estandarizados. Es especialmente útil para analizar tendencias históricas y generar reportes consistentes que apoyen decisiones basadas en información consolidada.

Por otro lado, un lago de datos es la elección apropiada cuando se trabaja con grandes volúmenes de datos no estructurados o se requiere realizar análisis avanzados como inteligencia artificial o aprendizaje automático. También es beneficioso cuando se desea

almacenar datos en su formato original para usos futuros sin necesidad de procesarlos previamente.

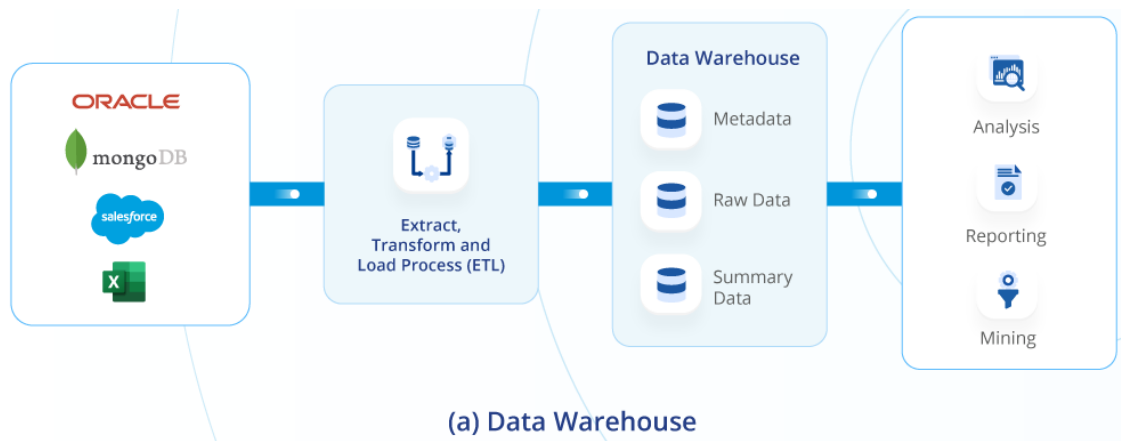


Figura 5. Almacén de datos - Datos estructurados

Fuente: www.astera.com

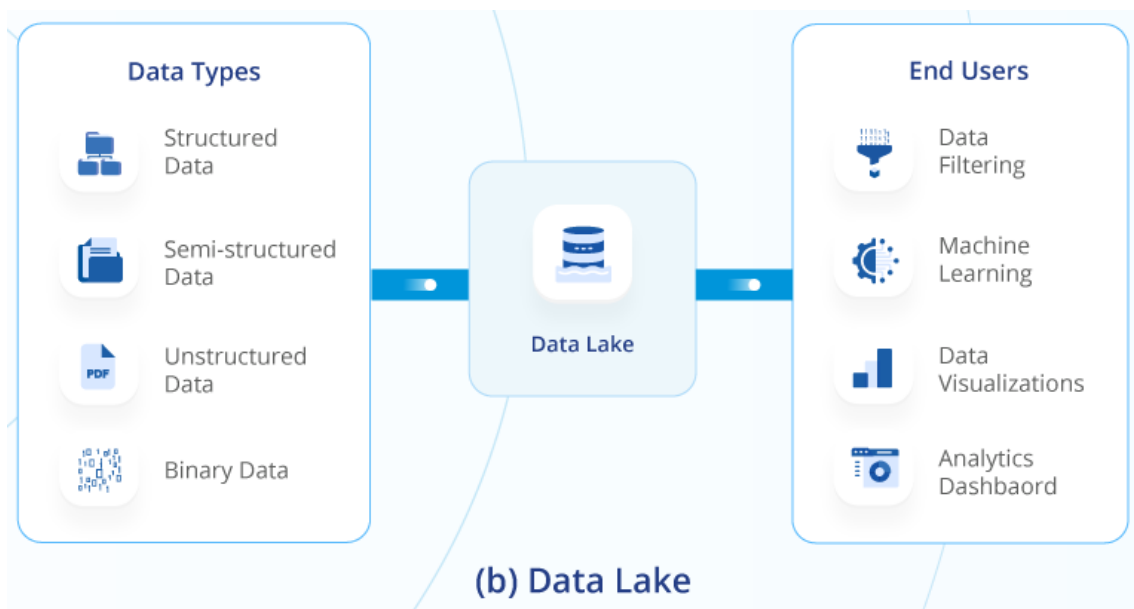


Figura 6. Lago de datos – Datos semi-estructurados y no estructurados

Fuente: www.astera.com

1.3. Evolución del almacenamiento de datos

El mundo del almacenamiento de datos ha evolucionado significativamente, impulsado por la creciente necesidad de manejar grandes volúmenes de información y la diversidad de formatos disponibles. Esta evolución ha llevado a la adopción de nuevas tecnologías y enfoques que permiten a las organizaciones adaptarse a las demandas cambiantes del mercado.

1.3.1. De las bases de datos relacionales tradicionales (SQL) a los enfoques modernos (NoSQL, Data Lakes)

Las **bases de datos relacionales** tradicionales, que utilizan SQL (Structured Query Language), han sido durante mucho tiempo el pilar del almacenamiento de datos. Son eficientes para manejar datos estructurados y operaciones transaccionales, pero presentan limitaciones cuando se trata de escalar horizontalmente o manejar datos no estructurados.

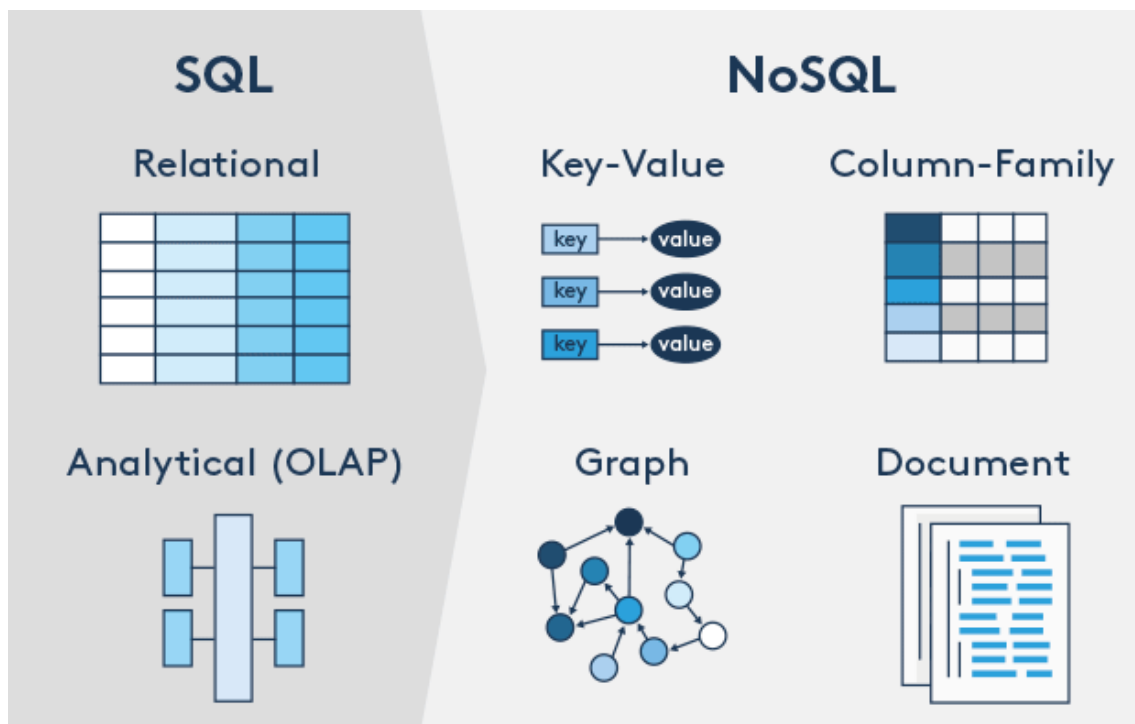


Figura 7. SQL vs. NoSQL

Fuente: www.certia.net

Con el auge del Big Data y la necesidad de procesar información no estructurada, surgieron las **bases de datos NoSQL**. Estas ofrecen mayor flexibilidad y escalabilidad, permitiendo almacenar y gestionar diferentes tipos de datos sin la rigidez de un esquema predefinido. Existen varios tipos de bases de datos NoSQL, como las documentales (por ejemplo, MongoDB), las de clave-valor (como Redis), las columnar (Cassandra) y las de grafos (Neo4j). Estas tecnologías son ideales para aplicaciones que requieren manejar grandes volúmenes de datos en tiempo real y ofrecer una alta disponibilidad.

Los **lagos de datos** representan otro paso en esta evolución, proporcionando una solución para almacenar datos en su forma bruta y sin restricciones de formato. Esto es especialmente útil para análisis avanzados y procesamiento masivo de datos, permitiendo a las organizaciones explorar y extraer valor de su información de maneras antes impensables.

1.3.2. Creciente demanda por manejo de datos no estructurados

El incremento en la generación de datos no estructurados, como imágenes, videos, registros de sensores y publicaciones en redes sociales, ha llevado a una mayor demanda de soluciones que puedan manejar esta diversidad de información. Las organizaciones reconocen el valor potencial de estos datos para obtener insights profundos y ventajas competitivas.

Las tecnologías modernas, como las bases de datos NoSQL y los lagos de datos, permiten a las empresas almacenar y procesar eficientemente datos no estructurados. Esto abre posibilidades en campos como el análisis predictivo, la inteligencia artificial y la automatización, donde el aprovechamiento de estos datos puede conducir a innovaciones significativas y mejoras en los procesos.

En conclusión, la evolución en el almacenamiento de datos refleja la necesidad de adaptarse a un entorno en constante cambio. Las organizaciones que adoptan estas nuevas tecnologías y enfoques están mejor posicionadas para aprovechar al máximo sus datos, tomar decisiones informadas y liderar en sus respectivos mercados.

2. Diseño de arquitecturas de almacenamiento

En el mundo actual, donde la información es uno de los activos más valiosos para las organizaciones, el diseño de arquitecturas de almacenamiento de datos se ha vuelto esencial. No se trata solo de guardar datos, sino de hacerlo de una manera que permita su acceso rápido, su análisis eficiente y su gestión segura. Este proceso implica planificar y crear estructuras que faciliten la recopilación, organización y almacenamiento de grandes volúmenes de información, garantizando que estén disponibles cuando se necesiten.

Imagina una biblioteca gigantesca. No basta con tener todos los libros; es crucial que estén ordenados de tal forma que puedas encontrar el que buscas sin perder tiempo. De la misma manera, una arquitectura de almacenamiento de datos bien diseñada debe ser escalable para crecer con la organización, flexible para adaptarse a nuevas necesidades y capaz de soportar tanto las operaciones diarias como los análisis más complejos.

2.1. Modelos de diseño en Almacenes de datos: Enfoques Kimball e Inmon

En el ámbito de los almacenes de datos, dos enfoques han predominado y se han convertido en referencia para profesionales y organizaciones: el de Ralph Kimball y el de Bill Inmon. Ambos expertos ofrecen estrategias distintas para estructurar los datos, cada una con sus propias ventajas y consideraciones. Entender estos modelos es fundamental para decidir cuál se ajusta mejor a las necesidades específicas de una organización.

2.1.1. Introducción a los dos modelos de diseño de Almacenes de datos más utilizados

- **Enfoque de Ralph Kimball:** Ralph Kimball es conocido por su enfoque práctico y orientado al usuario final. Su metodología, a menudo llamada enfoque dimensional o basado en áreas temáticas, se centra en diseñar almacenes de datos que sean intuitivos y fáciles de utilizar. La idea es que los datos estén organizados de manera que los analistas y responsables de la toma de decisiones puedan acceder a ellos y entenderlos sin dificultades, facilitando así el análisis multidimensional y la generación de insights valiosos. Este enfoque favorece

implementaciones rápidas y es especialmente útil cuando se necesita agilidad y flexibilidad.

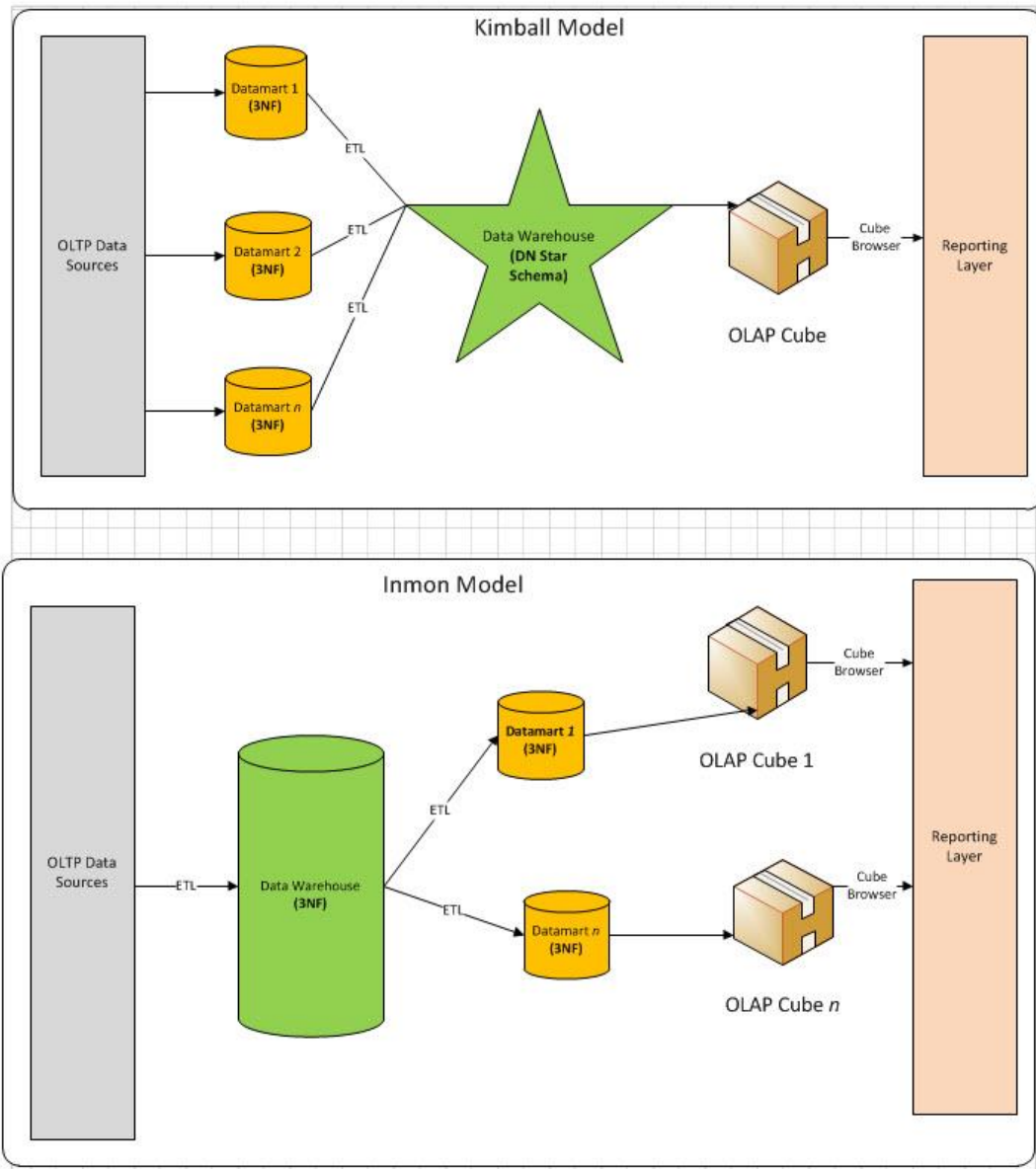


Figura 8. Enfoques Ralph Kimball vs. Bill Inmon

Fuente: bennyaustrin.com

- **Enfoque de Bill Inmon:** Bill Inmon, a menudo considerado el "padre del Data Warehousing", propone un enfoque más estructurado y centralizado, conocido como Corporate Information Factory (CIF). Su modelo se enfoca en construir un almacén de datos corporativo que actúe como la única fuente de verdad para toda

la organización. Esto significa que los datos están altamente normalizados y estructurados, proporcionando una visión integral y consistente de la información. Aunque puede requerir más tiempo y esfuerzo para implementarse, este enfoque asegura que todos en la organización trabajen con datos confiables y coherentes.

2.1.2. Enfoque de Ralph Kimball

Profundicemos en el enfoque de Kimball para entender cómo funciona y por qué es tan popular entre muchas empresas.

- **Diseño Dimensional y por Áreas Temáticas:** El corazón del enfoque de Kimball es el diseño dimensional. Imagina que estás organizando tu casa y decides asignar un lugar específico para cada tipo de objeto: ropa en el armario, libros en la estantería, utensilios en la cocina. De manera similar, Kimball propone crear **data marts**, que son subconjuntos del almacén de datos enfocados en áreas temáticas específicas como ventas, finanzas o recursos humanos.

Cada data mart está diseñado para ser fácilmente accesible y comprensible por los usuarios que lo necesitan. Esto facilita el análisis y la toma de decisiones, ya que la información relevante está organizada de forma lógica y clara. Además, la arquitectura es modular, lo que significa que puedes agregar nuevos data marts conforme surjan nuevas necesidades, sin tener que rehacer todo el sistema.

- **Tablas de Hechos y Dimensiones:** En este modelo, los datos se organizan en dos tipos principales de tablas:
 - **Tablas de Hechos:** Contienen datos cuantitativos que representan eventos o transacciones, como el total de ventas, número de unidades vendidas o costos. Son el núcleo del análisis y suelen estar en el centro del modelo dimensional.
 - **Tablas de Dimensiones:** Proporcionan contexto a las tablas de hechos. Incluyen información descriptiva como fechas, ubicaciones, productos o clientes. Estas tablas permiten analizar los hechos desde diferentes perspectivas y niveles de detalle.

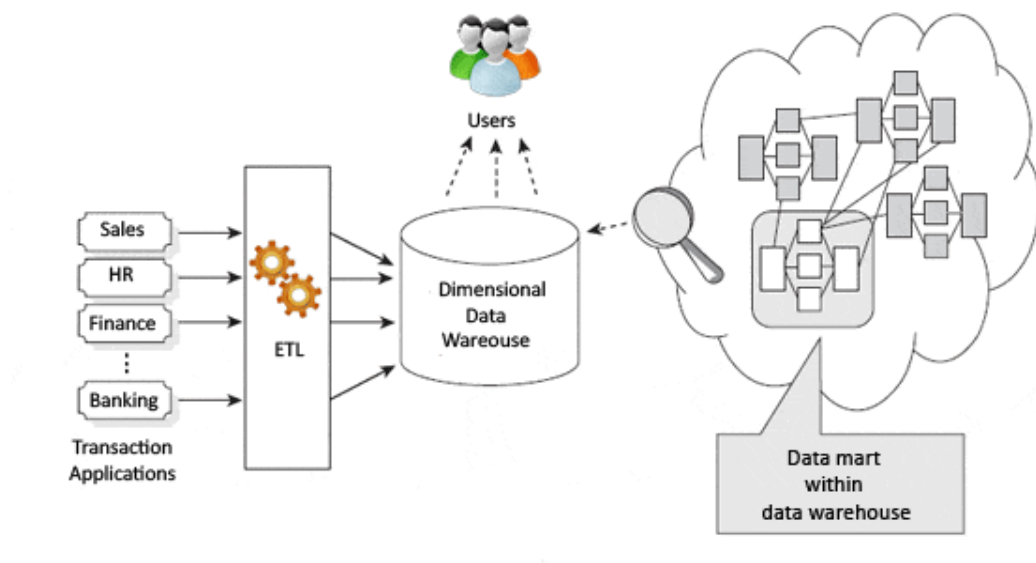


Figura 9. Enfoque Ralph Kimball

Fuente: www.astera.com

- **Esquemas Estrella vs. Esquemas Copo de Nieve:** Para estructurar estas tablas, se utilizan dos tipos de esquemas:
 - **Esquema Estrella:** En este diseño, la tabla de hechos está en el centro y se conecta directamente con las tablas de dimensiones, formando una estructura que se asemeja a una estrella. Es sencillo y facilita consultas rápidas, ya que requiere menos uniones (joins) entre tablas. Por ejemplo, una tabla de hechos de ventas podría conectarse directamente con tablas de dimensiones de productos, fechas y tiendas.
 - **Esquema Copo de Nieve:** Este es una extensión del esquema estrella, donde las tablas de dimensiones están normalizadas y se dividen en subtablas adicionales para eliminar redundancias. Aunque esto puede hacer que las consultas sean un poco más complejas y requieran más joins, mejora la organización y reduce la duplicación de datos. Por ejemplo, la tabla de dimensión de productos podría dividirse en subtablas de categorías y marcas.

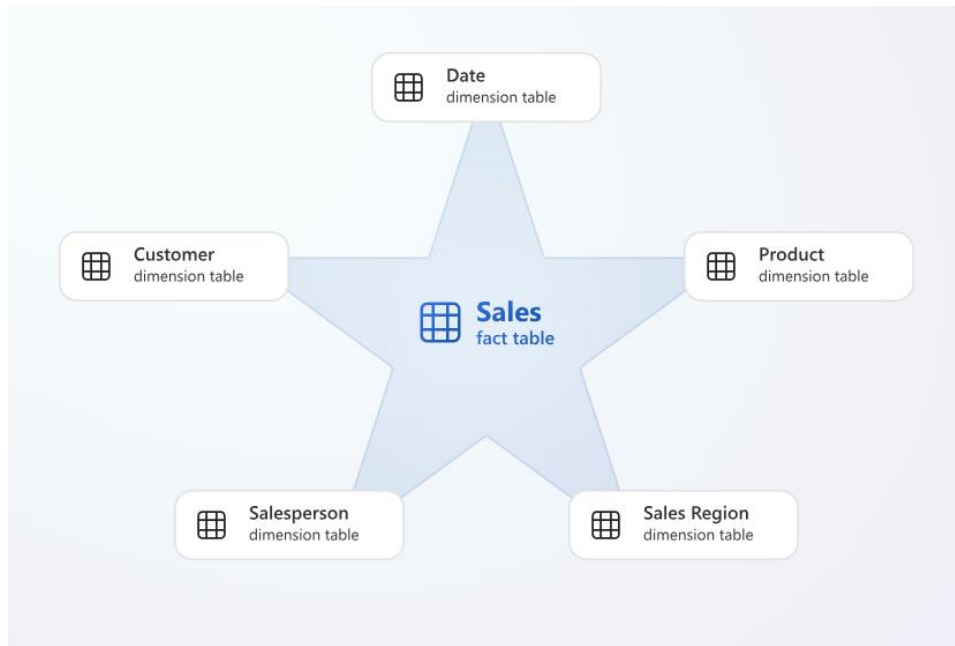


Figura 10. Esquema estrella

Fuente: learn.microsoft.com

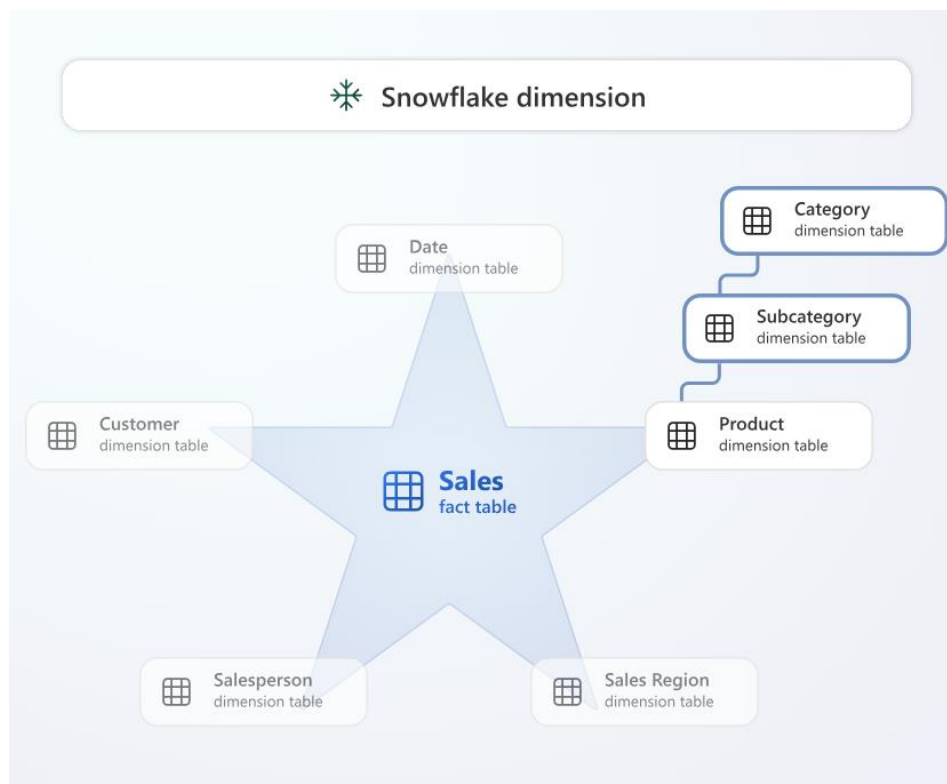


Figura 11. Esquema copo de nieve

Fuente: learn.microsoft.com

- **Ejemplos Prácticos:** Supongamos que trabajas en una empresa minorista y quieres analizar las ventas. Siguiendo el enfoque de Kimball:
 - La **tabla de hechos** almacenaría datos como el total de ventas, cantidad de productos vendidos y descuentos aplicados.
 - Las **tablas de dimensiones** podrían incluir:
 - **Productos:** detalles como nombre, categoría, marca.
 - **Tiempo:** fecha de venta, trimestre, año.
 - **Ubicación:** tienda, ciudad, región.

En un **esquema estrella**, todas estas dimensiones se conectarían directamente con la tabla de hechos. Si optas por un **esquema copo de nieve**, podrías descomponer la dimensión de productos en subdimensiones como categorías y marcas para eliminar redundancias.

Este enfoque facilita a los analistas entender y navegar por los datos, permitiendo realizar consultas complejas de manera eficiente y obtener información valiosa para la toma de decisiones.

2.1.3. Enfoque de Bill Inmon

Pasemos ahora al enfoque de Bill Inmon para comprender cómo difiere y qué beneficios ofrece.

- **Almacén de Datos Corporativo (Corporate Information Factory):** Inmon propone construir un almacén de datos centralizado que actúe como la única fuente de verdad para toda la organización. Esto significa integrar datos de todas las áreas de la empresa en una estructura coherente y altamente normalizada. La idea es que, independientemente del departamento o función, todos utilicen los mismos datos consistentes y actualizados.

Al normalizar los datos desde el principio, se eliminan redundancias y se asegura la integridad de la información. Este enfoque es especialmente útil para

organizaciones que necesitan una visión global y unificada de sus datos, como grandes corporaciones con múltiples divisiones y operaciones internacionales.

- **Bases de Datos Normalizadas para Consolidación de Datos:** La normalización implica dividir los datos en múltiples tablas relacionadas para evitar duplicaciones. Aunque esto puede hacer que la estructura sea más compleja y que las consultas requieran más joins, garantiza que los datos sean precisos y coherentes en todo momento.

Una vez que el almacén de datos corporativo está establecido, es posible crear data marts departamentales o temáticos a partir de él. Estos data marts pueden estar desnormalizados para mejorar el rendimiento de consultas específicas, pero al derivarse del almacén central, mantienen la coherencia y consistencia de los datos.



Figura 12. Enfoque Bill Inmon

Fuente: churriwifi.wordpress.com

- **Comparación con el Enfoque Kimball:**
 - **Centralización vs. Descentralización:** Inmon favorece una estructura centralizada, mientras que Kimball opta por una descentralización a través de data marts independientes.

- **Normalización vs. Desnormalización:** Inmon prioriza la normalización para mantener la integridad de los datos, mientras que Kimball utiliza la desnormalización para mejorar el rendimiento y facilitar el acceso.
- **Tiempo y Esfuerzo de Implementación:** El enfoque de Inmon suele requerir más tiempo y recursos para implementarse, ya que implica construir una estructura completa y centralizada. Sin embargo, ofrece una visión integral y consistente. El enfoque de Kimball permite implementaciones más rápidas y enfocadas en áreas específicas, brindando resultados inmediatos.

Comparación entre los enfoques:

Aspecto	Enfoque de Kimball	Enfoque de Inmon
Estructura	Descentralizada, basada en áreas temáticas	Centralizada, almacén de datos corporativo
Normalización	Desnormalización para rendimiento	Normalización para integridad
Facilidad de Uso	Fácil de entender y acceder para usuarios	Más complejo, pero integral
Escalabilidad	Modular, crece por áreas temáticas	Escalable a nivel corporativo

La elección entre ambos enfoques depende de las necesidades y prioridades de la organización. Algunas empresas pueden beneficiarse de la rapidez y flexibilidad del enfoque de Kimball, mientras que otras pueden preferir la consistencia y visión unificada que ofrece el enfoque de Inmon.

2.2. OLTP (Online Transaction Processing) vs. OLAP (Online Analytical Processing)

Al diseñar sistemas de procesamiento de datos, es fundamental entender la diferencia entre OLTP y OLAP, ya que tienen propósitos y requerimientos distintos.

2.2.1. Diferencias y similitudes en cuanto a diseño, enfoque y uso

- **OLTP (Online Transaction Processing):**
 - **Propósito:** Manejar operaciones transaccionales en tiempo real. Esto incluye actividades como ventas, pagos, reservas y cualquier transacción que requiera procesamiento inmediato y preciso.
 - **Diseño:** Utiliza bases de datos altamente normalizadas para asegurar la integridad de los datos y minimizar redundancias. La normalización ayuda a evitar inconsistencias y errores en los datos transaccionales.
 - **Uso:** Aplicaciones donde la velocidad y eficiencia en la gestión de datos es crítica, como sistemas bancarios, reservas de vuelos o puntos de venta en tiendas.
 - **Enfoque:** Optimizado para operaciones de lectura y escritura de alto rendimiento, con muchas transacciones concurrentes. La prioridad es que cada transacción se complete correctamente y los datos se actualicen en tiempo real.
- **OLAP (Online Analytical Processing):**
 - **Propósito:** Realizar análisis complejos y consultas multidimensionales en grandes volúmenes de datos, generalmente históricos. Ayuda a las organizaciones a entender tendencias, patrones y a tomar decisiones informadas.

- **Diseño:** Utiliza bases de datos desnormalizadas para facilitar la consulta rápida y eficiente. Esto permite acceder y analizar datos desde diferentes perspectivas sin que el rendimiento se vea afectado.
- **Uso:** Aplicaciones de análisis de negocios, generación de informes, minería de datos y cualquier actividad que requiera explorar grandes conjuntos de datos.
- **Enfoque:** Optimizado para operaciones de lectura intensiva. Las consultas pueden ser complejas y abarcar grandes volúmenes de datos, por lo que el diseño prioriza la eficiencia en la lectura sobre la velocidad de escritura.

2.2.2. Requisitos de rendimiento y cómo afectan el diseño arquitectónico

Los requisitos de rendimiento de OLTP y OLAP tienen un impacto directo en cómo se diseñan las bases de datos:

- **OLTP:**
 - **Alta velocidad en escritura y actualización:** Las transacciones deben procesarse en tiempo real, lo que requiere un sistema capaz de manejar múltiples operaciones simultáneamente sin retrasos.
 - **Integridad y consistencia de datos:** Es crucial que los datos sean precisos y estén actualizados al instante, ya que cualquier error puede tener consecuencias significativas.
 - **Diseño arquitectónico:** Se prioriza la normalización para evitar redundancias y asegurar que los datos sean consistentes. Esto puede resultar en más tablas y relaciones, pero es necesario para mantener la integridad.

- **OLAP:**
 - **Alta velocidad en lectura y consulta:** El sistema debe ser capaz de procesar consultas complejas y acceder a grandes volúmenes de datos de manera eficiente.
 - **Optimización para análisis:** Se desnormalizan las bases de datos para reducir la complejidad de las consultas y mejorar el rendimiento.
 - **Diseño arquitectónico:** Se utilizan esquemas como estrella o copo de nieve para estructurar los datos de forma que las consultas sean más rápidas y eficientes.

Entender estas diferencias es esencial para diseñar sistemas que cumplan con los objetivos específicos de la organización, ya sea para operaciones diarias o para análisis estratégicos.

2.3. ETL (Extracción, Transformación y Carga) vs. ELT (Extracción, Carga y Transformación)

En la gestión de datos, no solo es importante cómo se almacenan, sino también cómo se preparan para su uso. Aquí es donde entran en juego los procesos de ETL y ELT.

- **ETL (Extracción, Transformación y Carga):**
 - **Descripción:** Este es el proceso tradicional en el que los datos se extraen de diversas fuentes, se transforman para cumplir con los requisitos del almacén de datos (como limpieza, formateo y consolidación) y luego se cargan en el sistema final.
 - **Ventajas:**
 - **Calidad de datos:** Al transformar los datos antes de cargarlos, se asegura que solo información limpia y coherente llegue al almacén de datos.

- **Eficiencia en consultas:** Los datos ya están estructurados y optimizados para el análisis, lo que mejora el rendimiento.
- **Ejemplo:** Una empresa extrae datos de sus sistemas de ventas, marketing y atención al cliente, los transforma para unificarlos y corregir inconsistencias, y luego los carga en su almacén de datos para análisis y generación de informes.
- **ELT (Extracción, Carga y Transformación):**
 - **Descripción:** En este enfoque, los datos se extraen y cargan directamente en el almacén o lago de datos en su forma bruta. La transformación se realiza después, cuando los datos ya están almacenados y se necesitan para un análisis específico.
 - **Ventajas:**
 - **Flexibilidad y rapidez:** Permite almacenar rápidamente grandes volúmenes de datos sin necesidad de procesamiento previo.
 - **Adecuado para Big Data:** Es ideal cuando se trabaja con datos no estructurados o semi-estructurados y se necesita escalabilidad.
 - **Ejemplo:** Una organización recopila datos de sensores IoT y los almacena directamente en un lago de datos. Cuando necesitan analizar ciertos patrones o comportamientos, aplican las transformaciones necesarias en ese momento.

2.3.1. Importancia en los Almacenes de datos

La elección entre ETL y ELT depende de las necesidades y capacidades de la organización:

- **ETL:**
 - **Control de calidad:** Al transformar los datos antes de cargarlos, se garantiza que el almacén de datos mantenga un alto nivel de calidad y consistencia.
 - **Rendimiento en análisis:** Los datos ya están listos para ser analizados, lo que puede acelerar el proceso de generación de informes y toma de decisiones.
 - **Requiere planificación:** Es necesario definir claramente cómo se transformarán los datos, lo que puede requerir más tiempo y recursos.
- **ELT:**
 - **Rapidez en la ingesta de datos:** Al cargar los datos en bruto, se puede manejar un gran volumen de información sin retrasos.
 - **Flexibilidad:** Permite adaptar las transformaciones según las necesidades del análisis, lo que es útil en entornos dinámicos.
 - **Requiere capacidad de procesamiento:** La transformación en el momento del análisis puede requerir más potencia computacional y puede afectar el rendimiento si no se gestiona adecuadamente.

En conclusión, ETL es esencial para garantizar que los datos en el almacén sean de alta calidad y estén listos para el análisis, mientras que ELT ofrece una mayor flexibilidad y es adecuado para entornos donde los datos son variados y se necesita agilidad en su manejo.

3. Soluciones de almacenamiento en la nube

En la actualidad, el almacenamiento en la nube ha revolucionado la forma en que las organizaciones gestionan, procesan y acceden a sus datos. Gracias a estas soluciones, las empresas pueden disponer de una infraestructura flexible y escalable que les permite almacenar grandes volúmenes de información sin la necesidad de mantener y administrar hardware físico en sus propias instalaciones. Además, el modelo de pago por uso que ofrecen las nubes públicas permite ajustar los costos de almacenamiento según las necesidades reales de cada momento, optimizando así los recursos financieros. A continuación, exploraremos en detalle las ventajas del almacenamiento en la nube, los diferentes tipos disponibles y las consideraciones clave para su implementación.

3.1. Ventajas del almacenamiento en la nube

Las soluciones de almacenamiento en la nube presentan múltiples beneficios que las hacen especialmente atractivas para organizaciones de todos los tamaños y sectores. Entre las principales ventajas destacan la escalabilidad, la flexibilidad y los modelos de pago por uso, así como la reducción significativa de costos operativos.

3.1.1. Escalabilidad, flexibilidad y modelos de pago por uso

Una de las ventajas más destacadas del almacenamiento en la nube es su capacidad para escalar de manera automática y en función de las necesidades del usuario. Esto significa que las organizaciones pueden ajustar la cantidad de almacenamiento disponible a medida que sus datos crecen, sin tener que realizar grandes inversiones en hardware físico. Por ejemplo, si una empresa experimenta un aumento repentino en el volumen de datos debido a una campaña de marketing exitosa, puede ampliar su capacidad de almacenamiento en la nube en tiempo real, asegurando que no habrá interrupciones en el servicio.

La flexibilidad es otro aspecto clave. Las soluciones en la nube permiten a las organizaciones acceder a una amplia gama de servicios de almacenamiento, incluyendo bases de datos relacionales, NoSQL, lagos de datos y almacenes de datos, todo dentro de

un mismo entorno. Esto facilita la adopción de diferentes tecnologías según las necesidades específicas de cada proyecto o departamento. Por ejemplo, una empresa puede utilizar una base de datos SQL para sus operaciones transaccionales y, al mismo tiempo, emplear un lago de datos para almacenar y analizar grandes volúmenes de datos no estructurados provenientes de redes sociales o dispositivos IoT.

El modelo de pago por uso es otro beneficio significativo. En lugar de invertir grandes sumas en infraestructura que puede quedar infrautilizada, las organizaciones solo pagan por la capacidad de almacenamiento y los recursos que realmente utilizan. Esto permite una gestión más eficiente de los costos y facilita la planificación financiera. Además, este modelo ofrece la posibilidad de reducir gastos en épocas de menor actividad, ya que se puede disminuir el uso de recursos y, por ende, el costo asociado.

3.1.2. Reducción de costos operativos

El almacenamiento en la nube también contribuye a una disminución considerable de los costos operativos. Al eliminar la necesidad de mantener infraestructura física local, las empresas pueden ahorrar en gastos relacionados con la adquisición de hardware, mantenimiento, energía eléctrica y espacio físico. No es necesario contar con un equipo de TI dedicado a la gestión de servidores y sistemas de almacenamiento, lo que permite reasignar esos recursos a actividades más estratégicas para el negocio.

Además, el mantenimiento y las actualizaciones de software son gestionados directamente por el proveedor de servicios en la nube. Esto garantiza que las organizaciones siempre cuenten con las últimas actualizaciones de seguridad y mejoras en el rendimiento, sin incurrir en costos adicionales ni dedicar tiempo a estas tareas. La automatización de estos procesos minimiza el riesgo de errores humanos y mejora la eficiencia operativa.

3.2. Data Warehouses en la nube

Los almacenes de datos en la nube son soluciones diseñadas para almacenar, gestionar y analizar grandes volúmenes de datos estructurados. Estas plataformas permiten a las

organizaciones centralizar su información, facilitando la generación de informes y el análisis de datos para apoyar la toma de decisiones estratégicas. A diferencia de los sistemas tradicionales instalados en las instalaciones de la empresa (on-premise), los almacenes de datos en la nube ofrecen mayor flexibilidad, escalabilidad y no requieren inversiones iniciales significativas en infraestructura.

3.2.1. Soluciones como Amazon Redshift, Google BigQuery y Snowflake

Existen diversas plataformas líderes en el mercado que ofrecen servicios de almacenes de datos en la nube. Entre las más destacadas se encuentran:

- **Amazon Redshift:** Es una solución de almacén de datos completamente gestionada que forma parte de Amazon Web Services (AWS). Permite ejecutar consultas SQL analíticas sobre grandes volúmenes de datos estructurados, ofreciendo un rendimiento elevado y la capacidad de escalar según las necesidades. Amazon Redshift se integra fácilmente con otros servicios de AWS, como S3 para almacenamiento de datos no estructurados y Glue para procesos de ETL (Extracción, Transformación y Carga), lo que facilita la creación de un ecosistema completo para la gestión y análisis de datos.
- **Google BigQuery:** Es el almacén de datos serverless de Google Cloud Platform (GCP). BigQuery permite realizar análisis de grandes conjuntos de datos en tiempo real sin la necesidad de gestionar infraestructura. Una de sus principales ventajas es la capacidad de procesar consultas SQL de forma extremadamente rápida sobre petabytes de datos, lo que lo hace ideal para análisis avanzados y modelos predictivos. Además, BigQuery se integra de manera nativa con otras herramientas de Google, facilitando la colaboración y el flujo de datos.
- **Snowflake:** Es una plataforma de almacenamiento de datos en la nube independiente que ha ganado popularidad por su arquitectura única. Snowflake separa el almacenamiento del cómputo, lo que permite a las organizaciones escalar ambos de manera independiente según sus necesidades. Esto ofrece una gran flexibilidad y control de costos. Snowflake es compatible con datos

estructurados y semi-estructurados, y facilita la integración con diversas herramientas de análisis y procesamiento de datos.

3.2.2. Casos de uso y consideraciones al migrar a la nube

Los almacenes de datos en la nube son utilizados en una variedad de casos de uso:

- **Análisis de ventas y marketing:** Las empresas pueden consolidar datos de diferentes canales de venta y campañas de marketing para analizar el comportamiento de los clientes, identificar tendencias y optimizar estrategias comerciales.
- **Monitoreo del rendimiento empresarial:** Permiten realizar consultas complejas sobre operaciones internas, ayudando a identificar áreas de mejora y a tomar decisiones informadas basadas en indicadores clave de rendimiento (KPI).
- **Análisis financiero:** Facilitan la gestión y análisis de grandes volúmenes de transacciones financieras, soporte para auditorías y generación de proyecciones y presupuestos.

Al considerar la migración de un almacén de datos tradicional a la nube, es importante tener en cuenta varios aspectos:

- **Compatibilidad de datos:** Es esencial evaluar si los formatos de datos y sistemas actuales son compatibles con la plataforma en la nube elegida. Puede ser necesario transformar o adaptar ciertos datos para garantizar una migración exitosa.
- **Seguridad y cumplimiento normativo:** Al mover datos a la nube, especialmente información sensible, es fundamental asegurarse de que el proveedor cumpla con las regulaciones aplicables, como GDPR o HIPAA. También se deben implementar medidas de seguridad adecuadas para proteger los datos contra accesos no autorizados.
- **Costos de migración:** Aunque a largo plazo el almacenamiento en la nube puede resultar más económico, la migración inicial puede implicar costos significativos.

Es importante planificar y presupuestar adecuadamente, considerando el tiempo y los recursos necesarios para transferir y configurar los sistemas.

3.3. Lagos de datos en la nube

Los lagos de datos (Data Lakes) en la nube son repositorios que permiten almacenar grandes volúmenes de datos en su formato original y sin procesar. A diferencia de los almacenes de datos, que suelen contener información estructurada y procesada, los lagos de datos pueden almacenar datos estructurados, semi-estructurados y no estructurados. Esto los hace ideales para organizaciones que desean conservar todos sus datos para análisis futuros, incluyendo aquellos provenientes de fuentes como redes sociales, dispositivos IoT o registros de logs.

3.3.1. Implementación de soluciones como Azure Data Lake y Amazon S3

Varias plataformas en la nube ofrecen servicios de lagos de datos:

- **Azure Data Lake:** Es la solución de Microsoft para almacenar y analizar grandes volúmenes de datos no estructurados. Azure Data Lake es altamente escalable y se integra con otros servicios de Azure, como HDInsight y Azure Databricks, facilitando el procesamiento y análisis de datos mediante tecnologías como Hadoop y Spark. Esto permite a las organizaciones realizar análisis avanzados y desarrollar modelos de aprendizaje automático directamente sobre los datos almacenados.
- **Amazon S3 (Simple Storage Service):** Es uno de los servicios de almacenamiento en la nube más utilizados a nivel mundial. Amazon S3 permite almacenar y recuperar cualquier cantidad de datos en cualquier momento y desde cualquier lugar. Es altamente duradero y escalable, y soporta diferentes clases de almacenamiento para optimizar costos, como S3 Standard para acceso frecuente y S3 Glacier para archivado de datos a largo plazo. Amazon S3 se integra con una amplia gama de servicios de AWS, lo que facilita la creación de soluciones de análisis y procesamiento de datos completas.

3.3.2. Ventajas para almacenamiento de grandes volúmenes de datos no estructurados

Los lagos de datos en la nube ofrecen varias ventajas:

- **Costos reducidos:** Al permitir almacenar datos en su formato original sin necesidad de procesarlos previamente, se reducen los costos asociados con la transformación y estructuración de la información. Además, las diferentes clases de almacenamiento permiten optimizar los gastos en función de la frecuencia de acceso a los datos.
- **Flexibilidad y escalabilidad:** Los lagos de datos pueden crecer de manera prácticamente ilimitada, lo que es ideal para organizaciones que manejan grandes volúmenes de datos que aumentan con el tiempo. También permiten almacenar una amplia variedad de formatos de datos, lo que facilita su uso en diferentes tipos de análisis.
- **Acceso a datos no estructurados:** Permiten almacenar y analizar datos como imágenes, videos, registros de sensores y otros tipos de información que no encajan en modelos relacionales tradicionales. Esto abre oportunidades para realizar análisis más profundos y obtener insights que de otra manera serían difíciles de alcanzar.

3.4. Multinube y soluciones híbridas

En el contexto actual, muchas organizaciones optan por utilizar múltiples proveedores de servicios en la nube o combinar soluciones en la nube con infraestructura local, adoptando arquitecturas multinube e híbridas.

3.4.1. Uso de múltiples proveedores de nube y arquitecturas híbridas

- **Multinube:** Consiste en utilizar servicios de diferentes proveedores de nube pública, como AWS, Azure y Google Cloud, para aprovechar las fortalezas específicas de cada uno. Por ejemplo, una empresa puede utilizar Google Cloud

para sus capacidades de inteligencia artificial y aprendizaje automático, mientras que emplea AWS para almacenamiento y Azure para aplicaciones empresariales. Esto también ayuda a evitar la dependencia excesiva de un solo proveedor, reduciendo riesgos asociados con interrupciones de servicio o cambios en políticas y precios.

- **Soluciones híbridas:** Combinan infraestructura en la nube con sistemas locales (on-premise). Esto es útil cuando las organizaciones tienen aplicaciones o datos que, por razones de seguridad, cumplimiento normativo o rendimiento, deben permanecer en sus propias instalaciones. Al mismo tiempo, pueden aprovechar las ventajas de la nube para otras cargas de trabajo, logrando un equilibrio óptimo entre control y flexibilidad.

3.4.2. Gestión y optimización del rendimiento y costos

La adopción de arquitecturas multinube e híbridas requiere una gestión cuidadosa para maximizar beneficios y minimizar desafíos:

- **Optimización de costos:** Es importante monitorear constantemente el uso de recursos en diferentes plataformas para evitar gastos innecesarios. Herramientas de gestión y análisis de costos pueden ayudar a identificar oportunidades de ahorro y ajustar la asignación de recursos según las necesidades reales.
- **Gestión del rendimiento:** Distribuir cargas de trabajo entre diferentes nubes y entornos locales puede mejorar el rendimiento y la disponibilidad de los servicios. Sin embargo, es esencial asegurarse de que las aplicaciones y datos se integren adecuadamente y que la latencia y el ancho de banda no afecten negativamente a la experiencia del usuario.
- **Seguridad y cumplimiento:** Al manejar datos y aplicaciones en múltiples entornos, es crucial mantener estándares de seguridad consistentes y garantizar el cumplimiento de regulaciones en todas las plataformas. Esto puede implicar la implementación de políticas y herramientas de seguridad que funcionen de manera uniforme en diferentes nubes y sistemas locales.

4. Construcción de almacenes de datos

La construcción de un almacén de datos es esencial para las organizaciones que buscan transformar grandes volúmenes de información en conocimiento útil para la toma de decisiones estratégicas. Ahora que comprendemos los objetivos de un sistema de Almacén de Datos/Inteligencia de Negocios (DW/BI), es importante adentrarnos en los conceptos básicos del **modelado dimensional**, una técnica ampliamente aceptada que simplifica la estructura de las bases de datos y optimiza el rendimiento de las consultas.

El modelado dimensional aborda simultáneamente dos requisitos fundamentales: entrega datos que son comprensibles para los usuarios de negocio y ofrece un rendimiento rápido en las consultas. A lo largo de más de cinco décadas, organizaciones de TI, consultores y usuarios han favorecido estructuras dimensionales simples para satisfacer la necesidad humana de simplicidad. Esta simplicidad garantiza que los usuarios puedan entender fácilmente los datos y que el software pueda navegar y proporcionar resultados de manera eficiente.

Aunque los modelos dimensionales suelen implementarse en sistemas de gestión de bases de datos relacionales, difieren significativamente de los modelos normalizados en tercera forma normal (3NF) que buscan eliminar redundancias de datos. Mientras que las estructuras 3NF dividen los datos en muchas entidades discretas, convirtiéndolas en múltiples tablas relacionales, los modelos dimensionales priorizan la facilidad de uso y el rendimiento en las consultas.

Los modelos normalizados son extremadamente útiles en el procesamiento operativo, ya que una transacción de actualización o inserción afecta a la base de datos en un solo lugar. Sin embargo, pueden resultar demasiado complejos para las consultas de inteligencia de negocios (BI). Utilizar modelos normalizados en la capa de presentación de un sistema DW/BI dificulta la recuperación intuitiva y de alto rendimiento de los datos. Aquí es donde el modelado dimensional ofrece una solución al simplificar esquemas demasiado complejos.

4.1. Esquemas en estrella vs. Cubos OLAP

Los modelos dimensionales pueden implementarse de diferentes maneras, siendo las más comunes los **esquemas en estrella** y los **cubos OLAP (Online Analytical Processing)**. Ambos aprovechan los conceptos dimensionales, pero difieren en su implementación física y en sus capacidades.

Los **esquemas en estrella** se implementan en sistemas de gestión de bases de datos relacionales y reciben su nombre debido a su estructura que se asemeja a una estrella: una tabla central de hechos rodeada por tablas de dimensiones. Por otro lado, los **cubos OLAP** se implementan en entornos de bases de datos multidimensionales y permiten un análisis más sofisticado y rápido de los datos.

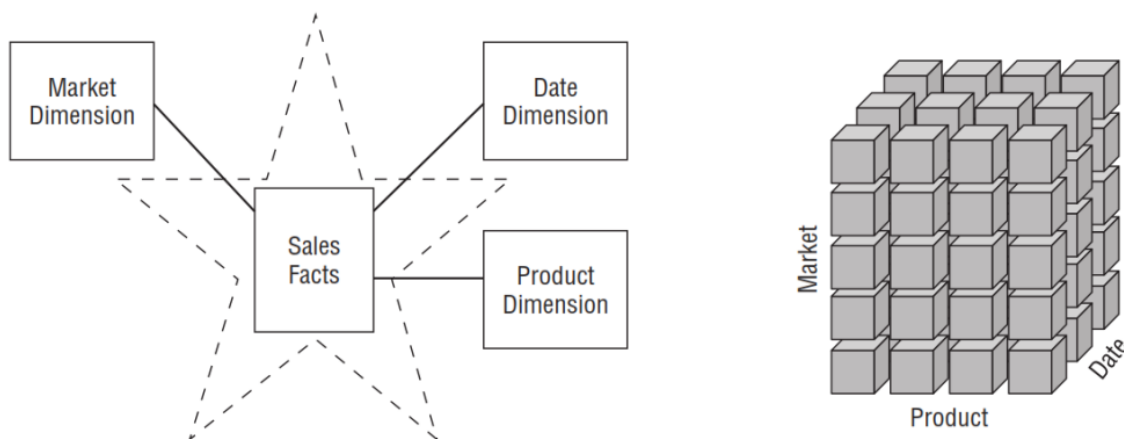


Figura 13. Esquema en estrella vs. Cubos OLAP

Fuente: The Data Warehouse Toolkit

Aunque ambos modelos comparten un diseño lógico común con dimensiones reconocibles, la implementación física difiere. Los cubos OLAP suelen crear y gestionar agregaciones de rendimiento o tablas de resumen precalculadas, lo que permite ofrecer un rendimiento de consulta superior gracias a cálculos previos, estrategias de indexación y otras optimizaciones. Sin embargo, esto puede resultar en un mayor tiempo y recursos necesarios durante la carga de datos, especialmente con grandes volúmenes de información.

Modelo Dimensional:

- **Adaptación del Modelo Relacional:** Es una adaptación del modelo de base de datos relacional tradicional, diseñado específicamente para mejorar el rendimiento en el análisis de datos y facilitar su comprensión por parte de los usuarios finales.
- **Estructura de Estrella o Copo de Nieve:** Los datos se organizan en una tabla de hechos central conectada a múltiples tablas de dimensiones, formando un esquema en estrella. En algunos casos, las dimensiones pueden estar normalizadas, formando un esquema de copo de nieve.
- **Enfoque en Consultas de Análisis:** Está orientado a optimizar la velocidad de las consultas de análisis de datos, facilitando el acceso rápido a grandes volúmenes de información para informes y análisis.

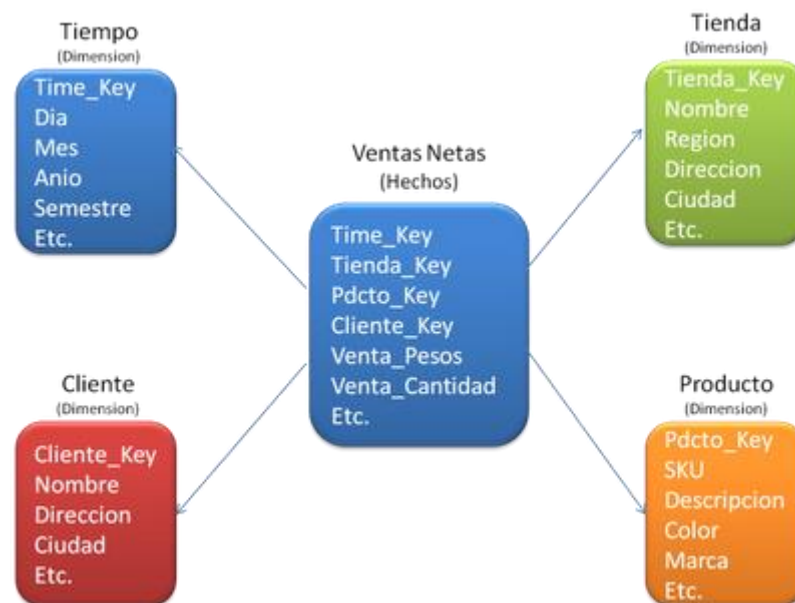


Figura 14. Modelo dimensional

Fuente: diegobazan7.wordpress.com

Cubo OLAP:

- **Implementación Propia:** Tiene una estructura interna propia, distinta de las bases de datos relacionales tradicionales. Puede basarse en datos almacenados en un sistema relacional (ROLAP), utilizar su propio almacenamiento especializado (MOLAP) o una combinación de ambos (HOLAP).
- **Representación Multidimensional:** Organiza los datos en una forma multidimensional, facilitando operaciones analíticas complejas y permitiendo a los usuarios explorar los datos desde múltiples perspectivas.
- **Análisis en Tiempo Real:** Está diseñado para consultas y análisis en tiempo real, proporcionando respuestas rápidas incluso en grandes conjuntos de datos.

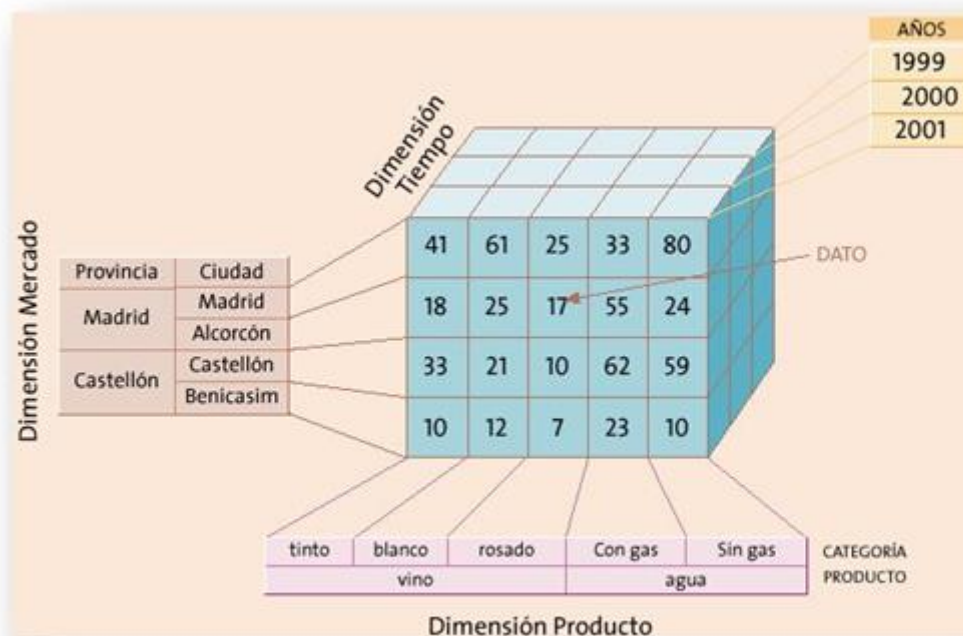


Figura 15. Cubo OLAP

Fuente: diegobazan7.wordpress.com

Ejemplos de plataformas que implementan cubos OLAP:

- Microsoft SQL Server Analysis Services (SSAS)
- Oracle OLAP
- IBM Cognos TM1
- SAP BW (Business Warehouse)
- MicroStrategy
- Pentaho
- Essbase de Oracle

4.2. Tablas de hechos para medir

La **tabla de hechos** es el núcleo del modelo dimensional y almacena las medidas cuantitativas resultantes de los eventos de los procesos de negocio de una organización. Estas medidas representan acciones específicas, como una venta, una transacción o un evento de producción. Es crucial que estas tablas no se repliquen en múltiples lugares para diferentes funciones organizacionales, garantizando así la consistencia en toda la empresa.

Por ejemplo, imagina que estás en una tienda minorista observando cómo se venden los productos. Cada vez que se escanea un producto en la caja registradora, se capturan medidas como la cantidad vendida y el monto de la venta. Cada fila de la tabla de hechos corresponde a un evento de medición y se encuentra en un nivel específico de detalle, llamado **grano**. Es fundamental que todas las filas de medidas en una tabla de hechos tengan el mismo grano, lo que significa que representan el mismo nivel de detalle.

Ejemplo de diseño de una tabla de hechos:

Supongamos una tienda minorista que registra sus ventas diarias por producto en cada tienda.

- **Nombre de la Tabla:** Ventas_Tienda
- **Grano:** Venta diaria por producto en cada tienda.

Columnas de la Tabla:

1. **ID_Venta:** Identificador único para cada registro de venta.
2. **Fecha_Venta:** Fecha en que se realizó la venta.
3. **ID_Tienda:** Identificador de la tienda donde se realizó la venta.
4. **ID_Producto:** Identificador del producto vendido.
5. **Cantidad_Vendida:** Número de unidades del producto vendidas.
6. **Ingresos:** Ingresos generados por la venta del producto.

Ejemplo de filas en la tabla:

ID_Venta	Fecha_Venta	ID_Tienda	ID_Producto	Cantidad_Vendida	Ingresos
1001	2024-01-10	5	200	3	150
1002	2024-01-10	5	204	2	80
1003	2024-01-10	3	200	1	50
...	...	...	...	...	...

Importancia de mantener el grano consistente:

No respetar el grano en una tabla de hechos puede generar varios problemas:

- **Inconsistencias en los datos:** Si diferentes filas tienen niveles de detalle distintos, se pueden producir discrepancias y confusiones al analizar la información.
- **Dificultades en el análisis y reportes:** Los usuarios pueden tener problemas para interpretar los resultados de sus consultas y realizar análisis significativos.
- **Problemas en la agregación de datos:** La suma o promedio de datos puede ser incorrecta si las medidas no están en el mismo nivel de detalle.
- **Riesgo de duplicación de datos:** Mezclar granos puede llevar a contar dos veces la misma información.

Tipos de datos en las medidas:

- **Datos aditivos:** Pueden sumarse en todas las dimensiones. *Ejemplo:* Ventas totales, unidades vendidas.
- **Datos semiaditivos:** Pueden sumarse en algunas dimensiones, pero no en todas, generalmente no a lo largo del tiempo. *Ejemplo:* Saldo bancario, inventario final.
- **Datos no aditivos:** No pueden sumarse de manera significativa en ninguna dimensión. *Ejemplo:* Porcentajes, ratios, precios unitarios.

Es importante evitar incluir filas con ceros que representen ninguna actividad (por ejemplo, días sin ventas) en la tabla de hechos, ya que esto aumentaría innecesariamente el tamaño de la tabla sin aportar información útil.

4.3. Tablas de dimensiones para describir el contexto

Las **tablas de dimensiones** complementan a la tabla de hechos al proporcionar el contexto descriptivo asociado con los eventos de medición. Contienen información textual que responde al "quién, qué, dónde, cuándo, cómo y por qué" de cada evento. Estas tablas suelen tener muchas columnas o atributos y pueden incluir detalles como el nombre del producto, la categoría, la marca, la fecha, la región, entre otros.

Características de las tablas de dimensiones:

- **Atributos detallados:** No es raro que una tabla de dimensión tenga entre 50 y 100 atributos, proporcionando información rica y detallada.
- **Clave principal única:** Cada dimensión tiene una clave primaria que la identifica de manera única y se utiliza para establecer relaciones con la tabla de hechos.
- **Relaciones jerárquicas:** Las dimensiones a menudo representan jerarquías, como productos que pertenecen a marcas y estas a categorías.

Ejemplo de una tabla de dimensión de productos:

ID_Producto	Nombre_Producto	Marca	Categoría	Precio	...
200	Camiseta X	Marca A	Ropa	50	
204	Pantalón Y	Marca B	Ropa	40	
210	Zapatos Z	Marca C	Calzado	80	
...	...	...	...	...	

Importancia de las tablas de dimensiones:

- **Facilitan el análisis:** Los atributos de las dimensiones sirven como fuente principal de restricciones de consulta, agrupaciones y etiquetas de informes. Por ejemplo, si un usuario desea ver las ventas por marca o categoría, la información debe estar disponible en la dimensión correspondiente.
- **Mejoran la usabilidad:** Los atributos deben consistir en palabras reales y evitar abreviaturas crípticas, facilitando la comprensión por parte de los usuarios.
- **Desnormalización intencional:** Aunque puede ser tentador normalizar las dimensiones (creando tablas separadas para marca y categoría), en el modelado dimensional se prefiere mantenerlas desnormalizadas para simplificar las consultas y mejorar el rendimiento.

4.4. Hechos y dimensiones unidos en un esquema de estrella

La unión de las tablas de hechos y dimensiones forma el **esquema en estrella**, donde la tabla de hechos está en el centro y las tablas de dimensiones la rodean. Esta estructura sencilla y simétrica facilita la comprensión y navegación de los datos por parte de los usuarios de negocio.

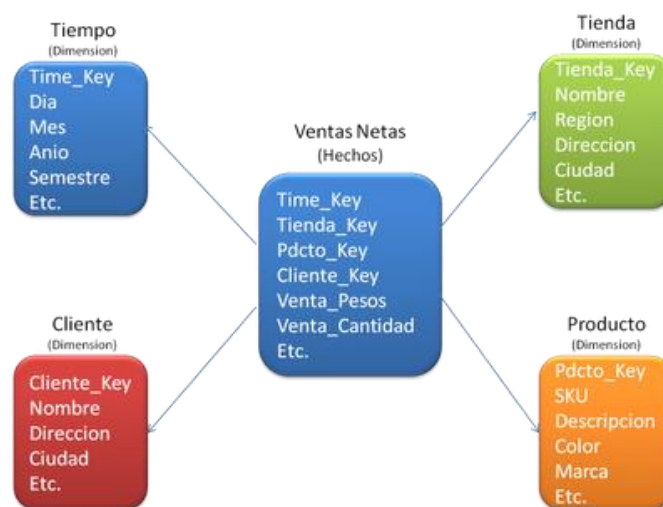


Figura 16. Modelo dimensional

Fuente: diegobazan7.wordpress.com

En este esquema:

- **Fact_Ventas:** Es la tabla de hechos que contiene las medidas cuantitativas (por ejemplo, cantidad vendida, ingresos).
- **Dim_Producto, Dim_Tienda, Dim_Fecha, Dim_Cliente:** Son tablas de dimensiones que proporcionan contexto a los hechos.

Beneficios del esquema en estrella:

- **Simplicidad:** La estructura es fácil de entender, lo que beneficia a los usuarios que necesitan realizar consultas y análisis.
- **Eficiencia en las consultas:** Los optimizadores de bases de datos pueden procesar estas estructuras con menos uniones de manera más eficiente, mejorando el rendimiento.

4.5. Conceptos generales asociados a los hechos

Existen varios conceptos importantes relacionados con las tablas de hechos que es necesario comprender para diseñar un almacén de datos efectivo.

4.5.1 Datos aditivos, semi-aditivos y no aditivos

En el análisis de datos y la construcción de modelos dimensionales, es fundamental entender cómo las medidas pueden ser agregadas a través de diferentes dimensiones. Las medidas se clasifican en tres categorías según su capacidad de ser sumadas: datos aditivos, semi-aditivos y no aditivos.

Datos aditivos

Los datos aditivos son aquellos que pueden ser sumados en todas las dimensiones de un modelo dimensional. Esto significa que, independientemente de cómo se agrupen o filtren los datos, la suma de estas medidas siempre tendrá un significado coherente y útil. Son los más sencillos de manejar y son especialmente valiosos en áreas como la contabilidad y el análisis de ventas, donde es crucial conocer los totales acumulados.

Por ejemplo, una empresa que desea analizar el total de ventas en una región específica, por mes o por producto, puede sumar los datos de ventas sin preocuparse por perder precisión o significado. Las ventas totales, los costos totales y la cantidad de productos vendidos son ejemplos de datos aditivos. Estas medidas pueden sumarse a través de diferentes fechas, regiones y productos, proporcionando información valiosa para la toma de decisiones.

Para ilustrar esto, consideremos la siguiente tabla:

Fecha	Región	Producto	Ventas	Unidades vendidas
2024-01-01	Norte	A	100	5
2024-01-01	Sur	A	200	10
2024-01-02	Norte	B	150	7
2024-01-02	Sur	B	250	12

En este ejemplo, tanto "Ventas" como "Unidades vendidas" son datos aditivos. Podemos sumar las ventas o las unidades vendidas por fecha, región o producto, y siempre obtendremos resultados significativos que reflejan el desempeño del negocio.

Datos semi-aditivos

Los datos semi-aditivos pueden ser sumados a través de algunas dimensiones, pero no en todas. Generalmente, no es apropiado sumarlos a lo largo de la dimensión de tiempo, ya

que hacerlo podría llevar a interpretaciones erróneas. Estos datos suelen representar valores de estado en un momento específico, como un saldo bancario o el nivel de inventario al final del día.

Por ejemplo, el saldo de una cuenta bancaria puede sumarse entre distintas cuentas o sucursales para obtener el total por sucursal o por cliente. Sin embargo, sumar el saldo de la misma cuenta en diferentes días no tiene sentido, ya que el saldo de cada día ya refleja el valor acumulado hasta ese momento. De manera similar, el inventario disponible al final del día puede sumarse por categoría de producto o tienda, pero no a lo largo del tiempo.

Consideremos la siguiente tabla:

Fecha	Sucursal	Cuenta	Saldo
2024-01-01	Sucursal 1	12345	5000
2024-01-01	Sucursal 2	67890	7000
2024-01-02	Sucursal 1	12345	5200
2024-01-02	Sucursal 2	67890	7100

En este caso, "Saldo" es un dato semi-aditivo. Podemos sumar los saldos entre diferentes sucursales o cuentas en una fecha determinada, pero no tiene sentido sumarlos a lo largo del tiempo. Sumar los saldos de varios días daría un valor incorrecto, ya que estaríamos agregando acumulaciones que ya incluyen los valores anteriores.

Datos no aditivos

Los datos no aditivos son aquellos que no pueden ser sumados en ninguna dimensión, ya que la suma no aporta un significado útil y puede llevar a conclusiones erróneas. En lugar de sumarlos, estos datos se analizan mediante promedios, máximos, mínimos u otros

cálculos más complejos. Suelen ser porcentajes, ratios o promedios que no tienen sentido al sumarlos directamente.

Por ejemplo, sumar los márgenes de ganancia de diferentes productos no es útil. En cambio, es más apropiado calcular un promedio ponderado que refleje el desempeño general. De igual manera, el porcentaje de ocupación de un hotel o la ratio de conversión en marketing requieren análisis específicos para obtener información valiosa.

Veamos la siguiente tabla:

Fecha	Producto	Margen de ganancia (%)
2024-01-01	A	20
2024-01-01	B	25
2024-01-02	A	15
2024-01-02	B	30

En este ejemplo, "Margen de ganancia (%)" es un dato no aditivo. Sumar el 20% del margen de ganancia del producto A con el 25% del producto B no tiene sentido y no proporcionaría una medida válida del rendimiento. En su lugar, deberíamos calcular el promedio ponderado de los márgenes, considerando factores como las ventas o las unidades vendidas, para obtener una visión general más precisa.

4.5.2 Valores nulos

En el contexto de las tablas de hechos en un modelo dimensional, un valor nulo representa la ausencia de un valor o datos desconocidos en una columna específica. Es importante destacar que un valor nulo no es lo mismo que un cero o un valor en blanco; un nulo indica que la información no está disponible o no se ha capturado. Los valores nulos pueden presentarse por diversas razones, como la falta de datos durante la captura, datos

que no aplican a ciertas combinaciones de dimensiones o errores en el proceso de carga de datos.

Impacto de los valores nulos en las tablas de hechos

Los valores nulos pueden afectar significativamente los cálculos agregados, como sumas, promedios o máximos, y pueden generar inexactitudes en los reportes y análisis. Dependiendo de cómo se manejen, pueden conducir a análisis distorsionados o conclusiones erróneas.

Consideremos la siguiente tabla de hechos con valores nulos:

Fecha	Producto	Región	Ventas	Costo
2024-01-01	A	Norte	100	NULL
2024-01-02	B	Sur	NULL	80
2024-01-03	C	Norte	150	90
2024-01-04	A	Sur	200	120

En este ejemplo:

- En el primer registro, el "Costo" es nulo, lo que indica que no se tenía disponible el costo asociado a esa venta.
- En el segundo registro, las "Ventas" son nulas, posiblemente porque la información no estaba disponible al cargar los datos.

Estrategias para tratar con valores nulos

Existen varias estrategias para abordar los valores nulos en las tablas de hechos, y la elección de la estrategia adecuada depende del contexto del negocio y del tipo de análisis que se desea realizar.

1. Eliminar registros con valores nulos

Una de las estrategias más simples es eliminar los registros que contienen valores nulos antes de realizar el análisis. Esto asegura que solo se consideren registros completos, evitando inexactitudes derivadas de datos faltantes. Esta opción es apropiada cuando los registros incompletos no aportan valor significativo y su ausencia no afecta los resultados. Sin embargo, eliminar registros puede llevar a la pérdida de información valiosa y sesgar el análisis si se eliminan demasiados datos.

2. Sustituir valores nulos con un valor por defecto

Otra estrategia es reemplazar los valores nulos por un valor por defecto, como cero. Esto evita "huecos" en los datos y permite continuar con el análisis sin interrupciones. Es esencial elegir el valor por defecto con cuidado para no distorsionar los resultados. Esta opción es útil cuando los valores nulos representan una situación específica que puede ser simbolizada por un valor estándar. No obstante, puede conducir a resultados potencialmente engañosos si los valores nulos representan datos importantes.

3. Imputación de valores faltantes

La imputación consiste en estimar los valores faltantes basándose en la información disponible. Se pueden utilizar métodos simples, como reemplazar con el promedio o la mediana, o técnicas más avanzadas como modelos predictivos. Esta estrategia es adecuada cuando se espera que los valores nulos puedan estimarse razonablemente y cuando eliminar registros resultaría en la pérdida de demasiados datos. Sin embargo, la imputación puede introducir sesgos si los valores imputados no representan adecuadamente los datos faltantes.

4. Mantener los valores nulos

En algunos casos, es preferible dejar los valores nulos intactos y ajustar el análisis para manejarlos adecuadamente. Esto puede ser útil cuando es importante registrar la falta de datos y no hacer suposiciones sobre ellos. Requiere que las consultas y los reportes estén configurados para manejar los nulos correctamente, evitando errores o interpretaciones incorrectas.

4.5.3 Year-To-Date (YTD)

Los usuarios de negocio a menudo requieren análisis de métricas acumuladas hasta la fecha, conocidas como Year-To-Date (YTD). Aunque puede parecer práctico almacenar directamente los valores acumulados de YTD en la tabla de hechos, esta práctica presenta varios inconvenientes y limitaciones.

¿Por qué no almacenar los datos YTD en la tabla de hechos?

Almacenar los valores acumulados de YTD en la tabla de hechos implica que, cada vez que se agregan nuevos datos, es necesario recalcular todos los valores acumulados de YTD. Esto puede ser costoso en términos de tiempo y recursos, especialmente en grandes volúmenes de datos. Además, almacenar YTD en la tabla de hechos limita la flexibilidad para ajustar el análisis según diferentes períodos, como años fiscales, trimestres o meses, lo que reduce la adaptabilidad necesaria en el análisis de negocios. También aumenta innecesariamente el tamaño de la tabla, generando duplicación de datos y potencialmente afectando el rendimiento de las consultas.

¿Por qué es mejor calcular YTD en las herramientas de BI o cubos OLAP?

Calcular las métricas de YTD en tiempo real utilizando herramientas de Business Intelligence (BI) o cubos OLAP ofrece varias ventajas:

- **Flexibilidad:** Permite definir dinámicamente qué constituye el "año" para el usuario, adaptándose a diferentes necesidades, como años fiscales o trimestres.
- **Consultas dinámicas:** Los usuarios pueden generar consultas ad-hoc y obtener diferentes vistas de los datos sin modificar la estructura de la base de datos.
- **Mejor rendimiento:** Los cálculos se realizan sobre los datos base cuando son necesarios, reduciendo la carga en la base de datos y mejorando la eficiencia.
- **Simplicidad en la tabla de hechos:** Al no almacenar valores acumulados, la tabla de hechos se mantiene más simple, almacenando únicamente los datos esenciales.

Ejemplo de cálculo de YTD en una aplicación de BI

Supongamos que tenemos una tabla de hechos que registra las ventas diarias por producto, región y fecha. Si un usuario desea calcular el YTD de las ventas hasta una fecha específica, la herramienta de BI puede sumar dinámicamente las ventas desde el inicio del año hasta la fecha solicitada.

Por ejemplo, para calcular las ventas acumuladas hasta el 1 de marzo de 2024:

Fecha	Producto	Región	Ventas
2024-01-01	Producto A	Norte	100
2024-01-02	Producto A	Norte	150
2024-01-03	Producto A	Norte	120
2024-02-01	Producto A	Norte	200

La herramienta sumaría las ventas de todas estas fechas, obteniendo un total de 870. Este cálculo se realiza en tiempo real, sin necesidad de almacenar los valores YTD en la tabla de hechos.

4.5.4 Tipos de tablas de hechos

En el diseño de un almacén de datos, las tablas de hechos son el componente central, ya que contienen las medidas cuantitativas que los usuarios desean analizar, como ventas, ingresos o costos. Estas medidas se pueden agregar y analizar a lo largo de múltiples dimensiones, como tiempo, producto o región. Dependiendo de los requisitos analíticos del negocio, existen varios tipos de tablas de hechos, cada una con un propósito específico.

1. Tabla de hechos transaccional

Las tablas de hechos transaccionales almacenan datos detallados sobre eventos individuales o transacciones que ocurren en un negocio. Cada fila representa una transacción única, como una venta o una compra. Son ideales para análisis a nivel granular, permitiendo investigar cada evento por separado.

Características:

- Nivel de detalle muy alto.
- Relacionadas con múltiples dimensiones: tiempo, producto, cliente, región.
- Se actualizan con cada nueva transacción.

Ejemplo:

Una tienda minorista registra cada venta realizada, incluyendo la fecha, el producto, el cliente y el monto.

Transacción ID	Fecha	Producto	Cliente	Región	Cantidad	Precio Unitario	Total Ventas
10001	2024-01-01	A	001	Norte	5	20	100
10002	2024-01-02	B	002	Sur	3	30	90
10003	2024-01-03	A	001	Norte	10	20	200

2. Tabla de hechos de snapshot periódico

Estas tablas capturan el estado de las medidas en intervalos de tiempo regulares, como diarios, semanales o mensuales. Son útiles para monitorear el estado o desempeño de una métrica en momentos específicos.

Características:

- Capturan el estado en momentos específicos.
- Incluyen dimensiones como tiempo y otras relevantes.
- Se actualizan en intervalos regulares.

Ejemplo:

Registro del saldo final de cuentas bancarias al final de cada día.

Fecha	Cuenta	Sucursal	Saldo Final del Día
2024-01-01	12345	Sucursal 1	5000
2024-01-02	12345	Sucursal 1	5200
2024-01-03	12345	Sucursal 1	5100

3. Tabla de hechos acumulativa

Estas tablas se utilizan para capturar el progreso de un proceso de negocio que tiene un inicio y un fin definidos, como el ciclo de vida de una orden de compra. Permiten monitorear procesos que pasan por varias etapas antes de completarse.

Características:

- Se centran en medir el progreso a través de distintas etapas.
- Incluyen dimensiones de tiempo para inicio y fin del proceso.
- Se actualizan a medida que el proceso avanza.

Ejemplo:

Seguimiento de una orden de compra desde su creación hasta su entrega.

Orden ID	Fecha Inicio	Fecha Envío	Fecha Entrega	Cliente	Producto	Estado de Orden	Costo Total
1001	2024-01-01	2024-01-05	2024-01-10	001	A	Completado	500
1002	2024-01-02	NULL	NULL	002	B	En proceso	300

4.5.5 Tablas de hechos sin hechos

Las tablas de hechos sin hechos, o "Factless Fact Tables", son un tipo especial de tabla en el modelado dimensional que no contienen medidas cuantitativas. Su función principal es registrar la ocurrencia de eventos o relaciones entre dimensiones de interés. Aunque no incluyen datos numéricos, son extremadamente útiles para ciertos tipos de análisis, como contar la frecuencia de eventos o verificar combinaciones de dimensiones.

Tipos de tablas de hechos sin hechos

1. Tablas de hechos sin hechos de eventos

Estas tablas registran la ocurrencia de un evento sin una medida cuantitativa asociada. Permiten contar cuántas veces ocurrió un evento y analizar su distribución a lo largo de diferentes dimensiones.

Ejemplo:

Registro de la asistencia de estudiantes a clases.

Fecha	Estudiante	Clase	Profesor	Asistencia
2024-01-01	1001	Mat101	ProfA	Sí
2024-01-02	1002	Mat101	ProfA	Sí
2024-01-01	1001	Fis101	ProfB	Sí
2024-01-03	1003	Mat101	ProfA	No

2. Tablas de hechos sin hechos de cobertura

Estas tablas representan relaciones entre dimensiones o verifican si una combinación específica de dimensiones existe en los datos. Son útiles para detectar situaciones faltantes, como productos no vendidos en una región.

Ejemplo:

Registro de qué productos están disponibles en qué tiendas.

Tienda	Producto	Región
Tienda1	ProdA	Norte
Tienda2	ProdB	Sur

4.5.6 Claves subrogadas

Las claves subrogadas son identificadores artificiales utilizados en las tablas de hechos y dimensiones dentro de un almacén de datos. En lugar de utilizar claves naturales provenientes de los sistemas transaccionales (como números de cliente o códigos de producto), se generan claves subrogadas para crear una identificación única y consistente de los registros. Esto permite manejar eficientemente las relaciones entre tablas y gestionar situaciones como la historia de cambios en las dimensiones o inconsistencias en las fuentes de datos.

¿Por qué usar claves subrogadas?

- **Desacoplamiento de los sistemas de origen:** Los sistemas transaccionales pueden cambiar sus claves naturales; las claves subrogadas hacen que el almacén de datos sea independiente de esos cambios.
- **Manejo de la historia de los datos:** Permiten distinguir entre diferentes versiones de un mismo registro, esencial para llevar un historial preciso.
- **Optimización del rendimiento:** Al ser números enteros pequeños, mejoran el rendimiento en las uniones entre tablas.
- **Manejo de datos faltantes o inconsistentes:** Ayudan a gestionar inconsistencias en las claves de los sistemas de origen sin afectar el análisis.

Ejemplo de uso de claves subrogadas

Supongamos una tabla de hechos de ventas:

Dimensión de Clientes:

Clave Subrogada Cliente	ID Cliente (Natural)	Nombre del Cliente	Ciudad
1	1001	Juan Pérez	Ciudad A
2	1002	María Gómez	Ciudad B

Tabla de Hechos de Ventas:

Clave Subrogada Fecha	Clave Subrogada Cliente	Clave Subrogada Producto	Cantidad	Total Ventas
1	1	2	2	600
2	2	1	1	400

En este escenario, las claves subrogadas permiten relacionar eficientemente las ventas con las dimensiones correspondientes, independientemente de cambios en los datos de origen.

4.6 Conceptos generales asociados a las dimensiones

En el diseño de modelos dimensionales para almacenes de datos, las dimensiones son elementos esenciales que proporcionan el contexto necesario para interpretar y analizar las medidas almacenadas en las tablas de hechos. Mientras que las medidas representan datos cuantitativos, como ventas, costos o ingresos, las dimensiones aportan atributos descriptivos que permiten entender y segmentar esos números desde diferentes perspectivas.

Imagina que tienes un conjunto de cifras de ventas. Por sí solas, estas cifras no ofrecen mucha información. Sin embargo, al relacionarlas con dimensiones como productos,

clientes, fechas o regiones, es posible analizar las ventas por categoría de producto, por segmento de clientes, por períodos específicos o por áreas geográficas. Las dimensiones, por tanto, enriquecen los datos y hacen posible el análisis detallado que necesitan las empresas para tomar decisiones informadas.

Claves primarias en las dimensiones

Cada tabla de dimensión cuenta con una clave primaria que identifica de forma única cada registro dentro de esa dimensión. Esta clave es fundamental porque permite establecer relaciones entre las tablas de hechos y las dimensiones. Por ejemplo, si tienes una dimensión de productos, la clave primaria podría ser un identificador único como un código de producto o un número generado por el sistema.

Considera la siguiente tabla de dimensión de productos:

Producto_ID	Nombre	Categoría
P001	Sunglasses TR-7	Accesorios
P002	Chocolate bar 70% cacao	Dulces
P003	Oatmeal biscuits	Dulces

En este caso, "Producto_ID" es la clave primaria que permite identificar de manera única cada producto. Esta clave es la que se utilizará en la tabla de hechos para relacionar las ventas con los productos correspondientes.

Uso de claves subrogadas

En el contexto de un modelo dimensional, es común utilizar claves subrogadas en lugar de claves naturales. Las claves subrogadas son identificadores artificiales, generalmente números enteros generados secuencialmente, que reemplazan a las claves naturales

provenientes de los sistemas transaccionales, como códigos de producto o números de cliente.

¿Por qué utilizar claves subrogadas?

1. **Desacoplamiento de sistemas transaccionales:** Las claves naturales pueden cambiar en los sistemas de origen, lo que dificultaría mantener la integridad de los datos en el almacén de datos. Al usar claves subrogadas, se logra independencia respecto a esos cambios, garantizando que las relaciones entre tablas se mantengan estables.
2. **Manejo de cambios históricos:** Las claves subrogadas permiten capturar diferentes versiones de una misma entidad. Por ejemplo, si un cliente cambia de dirección, cada versión de ese cliente puede tener su propia clave subrogada, facilitando el seguimiento de los cambios a lo largo del tiempo.
3. **Optimización de las consultas:** Las claves subrogadas suelen ser números enteros pequeños, lo que mejora la eficiencia de las consultas, especialmente al realizar uniones (joins) entre las tablas de hechos y dimensiones.

Siguiendo con el ejemplo anterior, la dimensión de productos con claves subrogadas quedaría así:

Producto_PK	Nombre	Categoría
1	Sunglasses TR-7	Accesorios
2	Chocolate bar 70% cacao	Dulces
3	Oatmeal biscuits	Dulces

Aquí, "Producto_PK" es la clave subrogada que reemplaza a "Producto_ID". En la tabla de hechos, se utilizará "Producto_PK" para referenciar los productos.

Tablas de consulta (Lookup Tables)

Las tablas de dimensión también se conocen como tablas de consulta o "lookup tables". Estas tablas almacenan información detallada de los atributos, pero la tabla de hechos no almacena todos estos atributos completos. En su lugar, la tabla de hechos contiene las claves que hacen referencia a las dimensiones, lo que reduce el tamaño de los datos y facilita las uniones durante el análisis.

Por ejemplo, en una tabla de hechos de ventas podrías tener:

Fecha	Producto_PK	Cantidad	Total_Ventas
2024-01-01	1	2	400
2024-01-02	2	3	600

Aquí, "Producto_PK" es la clave que relaciona cada venta con el producto correspondiente en la dimensión de productos. Si necesitas más información sobre el producto, como su nombre o categoría, puedes realizar una unión entre la tabla de hechos y la tabla de dimensión utilizando "Producto_PK".

Group & Filter: Slice and Dice

Una de las grandes ventajas de estructurar un modelo dimensional con dimensiones y tablas de hechos es la capacidad de realizar operaciones de "slice and dice". Este término se refiere a la posibilidad de agrupar y filtrar los datos desde diferentes perspectivas, permitiendo a los usuarios explorar la información de manera flexible y dinámica.

Por ejemplo, podrías querer analizar el monto total de ventas por producto. Utilizando las dimensiones y la tabla de hechos, podrías obtener:

Producto	Total Ventas
Sunglasses TR-7	\$3,543
Chocolate bar 70% cacao	\$24,678
Oatmeal biscuits	\$1,456

Esto te permite identificar cuáles productos tienen mejor desempeño y tomar decisiones estratégicas, como enfocarte en promociones o ajustar el inventario.

4.6.1 Dimensión Fecha

La dimensión de fecha es posiblemente una de las más importantes en cualquier modelo dimensional, ya que permite analizar los hechos y las medidas a lo largo del tiempo. No solo facilita el análisis diario, sino que también permite desglosar los datos por semanas, meses, trimestres y años. La mayoría de los análisis de negocio requieren una perspectiva temporal para entender tendencias, estacionalidades y comparar períodos.

¿Qué es la dimensión de fecha?

La dimensión de fecha es una tabla que contiene un registro para cada día dentro de un rango de años determinado, por ejemplo, de 2010 a 2030. Cada registro representa un día único y contiene varios atributos que describen diferentes aspectos de esa fecha, permitiendo realizar análisis detallados y flexibles.

Atributos comunes de la dimensión fecha

Los atributos en la dimensión de fecha pueden incluir:

- **Fecha completa:** La fecha específica, como "2024-01-01".

- **Día de la semana:** El nombre del día, como "lunes", o su número, como "1" para lunes.
- **Mes:** Nombre o número del mes, como "Enero" o "01".
- **Trimestre:** El trimestre del año, como "Q1" para el primer trimestre.
- **Año:** El año al que pertenece la fecha, como "2024".
- **Año fiscal:** Si el año fiscal difiere del calendario, este atributo es crucial.
- **Festivo:** Indica si la fecha corresponde a un día festivo, como "Navidad".

Por ejemplo, una tabla de dimensión fecha podría verse así:

Fecha_PK	Fecha Completa	Día Semana	Mes	Año	Trimestre	Día Festivo
1	2024-01-01	Lunes	Enero	2024	Q1	Año Nuevo
2	2024-01-02	Martes	Enero	2024	Q1	
3	2024-01-03	Miércoles	Enero	2024	Q1	

Uso de la dimensión de fecha en el análisis de datos

La dimensión de fecha es fundamental para realizar diversos tipos de análisis:

1. **Análisis de tendencias:** Permite observar cómo evolucionan las ventas u otras métricas a lo largo del tiempo, identificando patrones o anomalías.
2. **Comparaciones entre períodos:** Facilita la comparación de métricas entre diferentes períodos, como comparar las ventas del primer trimestre de 2024 con las del primer trimestre de 2023.

3. **Análisis por períodos fiscales:** Si la empresa opera con un año fiscal diferente al calendario, la dimensión fecha permite ajustar los análisis según el ciclo fiscal correcto.
4. **Identificación de patrones estacionales:** Ayuda a detectar estacionalidades, como aumentos de ventas en ciertos meses o días de la semana.
5. **Análisis de días laborables vs. no laborables:** Al incluir atributos que indican si un día es laborable o festivo, es posible analizar el impacto de estos días en las métricas del negocio.

Usar la fecha natural o una clave subrogada como clave primaria

Existen dos enfoques para definir la clave primaria en la dimensión de fecha:

1. **Usar la fecha natural como clave primaria:** Se utiliza la fecha misma, como "2024-01-01", como identificador único. Este enfoque es intuitivo y simplifica el diseño, pero puede tener implicaciones en el rendimiento al realizar uniones con la tabla de hechos, ya que los tipos de datos de fecha ocupan más espacio y pueden ser más lentos en las operaciones.
2. **Usar una clave subrogada:** Se asigna un número entero único a cada fecha, como "1" para "2024-01-01". Este método mejora el rendimiento en las uniones y es consistente con el uso de claves subrogadas en otras dimensiones. Aunque requiere un paso adicional en el proceso de carga de datos para generar las claves, ofrece ventajas en almacenes de datos de gran tamaño.

4.6.2 Nulos en dimensiones

En el diseño de modelos dimensionales, es común enfrentar situaciones donde algunos valores de los atributos en las dimensiones son desconocidos o no aplican, resultando en valores nulos. Manejar adecuadamente estos valores es crucial para mantener la integridad de los datos y asegurar que los análisis sean precisos y significativos.

¿Qué son los valores nulos en las dimensiones?

Un valor nulo en una dimensión indica la ausencia de información en un campo específico. Esto puede ocurrir por varias razones: falta de datos en la fuente, errores en la captura, o porque el atributo no es aplicable a ciertos registros. Por ejemplo, en una dimensión de clientes, es posible que no se tenga el número de teléfono de algunos clientes.

Problemas que generan los nulos en las dimensiones

Los valores nulos pueden causar varios inconvenientes:

- **Errores en las uniones:** Las uniones entre tablas de hechos y dimensiones pueden fallar o excluir registros si las claves foráneas contienen valores nulos.
- **Interpretaciones incorrectas:** Un valor nulo puede ser malinterpretado, llevando a conclusiones erróneas en los análisis.
- **Cálculos inexactos:** Los valores nulos pueden afectar operaciones como sumas, promedios o conteos, si no se manejan adecuadamente.

Estrategias para manejar los valores nulos en dimensiones

1. **Sustitución con valores por defecto:** Reemplazar los nulos con un valor predeterminado que indique claramente la falta de información, como "Desconocido" o "No aplica". Esto evita problemas en las uniones y facilita el análisis.

Ejemplo: Si un producto no tiene categoría asignada, en lugar de dejar el campo nulo, se asigna "Sin Categoría".

2. **Uso de claves subrogadas especiales:** Crear una entrada especial en la dimensión para representar datos desconocidos o no aplicables, utilizando una clave subrogada específica.

Ejemplo: En la dimensión de clientes, se agrega un registro con "Cliente_PK = 0" y "Nombre = Cliente Desconocido". Cuando falta información del cliente en la tabla de hechos, se usa esta clave.

3. **Eliminar o ignorar registros con nulos:** Si los datos faltantes son insignificantes, se pueden excluir estos registros del análisis. Sin embargo, hay que tener cuidado de no sesgar los resultados.
4. **Imputación de datos faltantes:** Estimar los valores nulos basándose en otros datos disponibles. Esta estrategia es útil cuando los valores faltantes son frecuentes y su estimación puede mejorar el análisis.

4.6.3 Jerarquías en dimensiones

Las jerarquías en dimensiones son estructuras que organizan los atributos de una dimensión en niveles, permitiendo analizar los datos desde niveles generales hasta detalles específicos. Facilitan la navegación y el análisis multidimensional, ofreciendo a los usuarios la capacidad de "perforar" en los datos para obtener insights más profundos.

¿Qué son las jerarquías en dimensiones?

Una jerarquía es una secuencia ordenada de atributos que reflejan una relación natural de superioridad y subordinación. Por ejemplo, en una dimensión geográfica, una jerarquía típica podría ser:

País → Estado/Provincia → Ciudad → Tienda

Esto permite a los usuarios analizar las ventas totales por país y luego desglosarlas por estado, ciudad y hasta nivel de tienda.

Tipos de jerarquías en dimensiones

1. **Jerarquías naturales (balanceadas):** Cada nivel está completamente definido por el nivel superior sin huecos. Todos los caminos desde el nivel superior al inferior tienen la misma longitud.



Figura 17. Jerarquías balanceadas

Fuente: ibm.com

2. **Jerarquías no balanceadas:** Algunos nodos tienen más niveles que otros. Por ejemplo, en una organización, algunos departamentos pueden tener más niveles jerárquicos que otros.

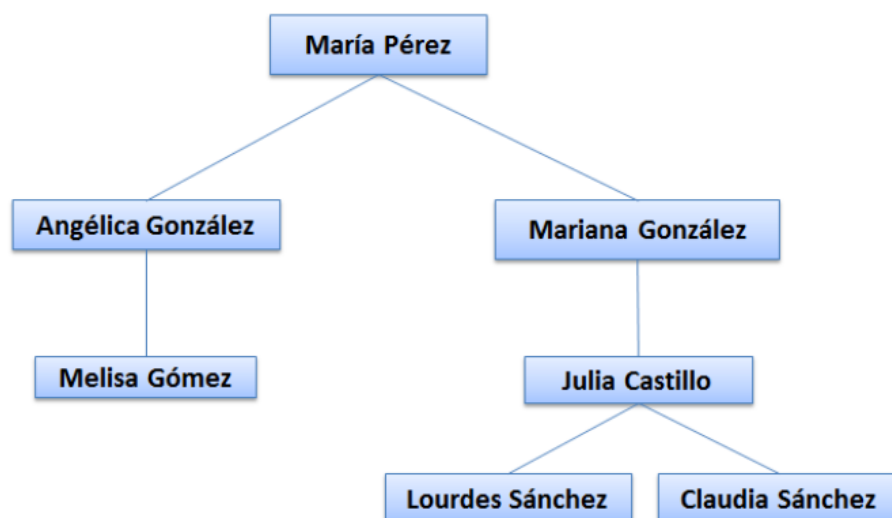


Figura 18. Jerarquías no balanceadas

Fuente: researchgate.net

3. **Jerarquías con niveles opcionales:** Existen niveles que pueden ser saltados. Por ejemplo, un producto puede estar asignado directamente a una categoría sin pasar por una subcategoría intermedia.



Figura 19. Jerarquía con niveles opcionales

Fuente: researchgate.net

Ventajas de las jerarquías en las dimensiones

- **Facilitan el análisis multinivel:** Permiten agregar y desagregar datos en diferentes niveles de detalle.
- **Mejoran la navegación de datos:** Los usuarios pueden explorar los datos de manera más intuitiva y descubrir patrones o anomalías.

Desafíos de las jerarquías

- **Manejo de datos faltantes o incompletos:** Las jerarquías no balanceadas o con niveles opcionales pueden complicar el análisis si no se manejan adecuadamente.
- **Rendimiento:** Jerarquías complejas en grandes volúmenes de datos pueden requerir optimizaciones para mantener un buen rendimiento.

4.6.4 Dimensiones conformadas

Las dimensiones conformadas son dimensiones que se comparten entre múltiples tablas de hechos o data marts dentro de un almacén de datos. Estas dimensiones están diseñadas de manera consistente para asegurar que los análisis sean coherentes y comparables a lo largo de diferentes áreas del negocio.

¿Qué son las dimensiones conformadas?

Son tablas de dimensión estandarizadas que mantienen la misma estructura y contenido en todos los lugares donde se utilizan. Esto permite que diferentes tablas de hechos puedan ser combinadas o comparadas utilizando estas dimensiones comunes.

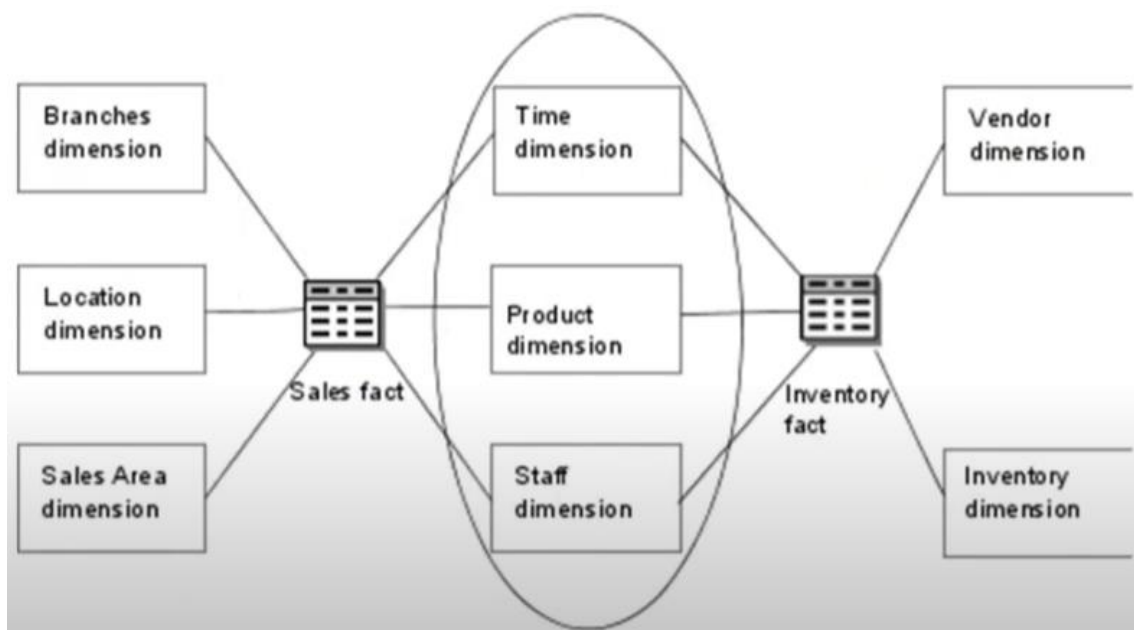


Figura 20. Dimensiones conformadas

Fuente: Youtube – Learn at Knowstar

Importancia de las dimensiones conformadas

- **Consistencia en el análisis:** Garantizan que las métricas se calculen e interpreten de la misma manera en toda la organización.

- **Facilitan la integración de datos:** Permiten combinar datos de diferentes áreas o sistemas de manera coherente.
- **Simplifican el mantenimiento:** Al tener una única versión de la dimensión, se reducen los esfuerzos de actualización y sincronización.

Ejemplos comunes de dimensiones conformadas

1. **Dimensión de tiempo:** Utilizada en múltiples tablas de hechos para asegurar que el análisis temporal sea consistente.
2. **Dimensión de producto:** Compartida entre ventas, inventarios y promociones para analizar el desempeño de los productos desde diferentes perspectivas.
3. **Dimensión de cliente:** Utilizada en ventas, marketing y servicio al cliente, permitiendo una visión unificada del cliente.

4.6.5 Dimensiones degeneradas

Las dimensiones degeneradas son atributos que, aunque son dimensiones conceptualmente, se almacenan directamente en la tabla de hechos sin tener una tabla de dimensión asociada. Generalmente, representan identificadores únicos de transacciones, como números de factura o pedido.

¿Qué son las dimensiones degeneradas?

Son atributos no numéricos que se incluyen en la tabla de hechos para permitir análisis detallados a nivel de transacción. No tienen atributos adicionales que justifiquen una tabla de dimensión independiente.

Características de las dimensiones degeneradas

- **No tienen tabla de dimensión:** Se almacenan directamente en la tabla de hechos.
- **Son identificadores únicos:** Como números de pedido, factura o transacción.
- **Relación uno a uno con la tabla de hechos:** Cada valor de la dimensión degenerada corresponde a una fila en la tabla de hechos.

Ejemplo

En una tabla de hechos de ventas:

Fecha	Cliente_PK	Producto_PK	Cantidad	Total Ventas	Número de Pedido
2024-01-01	123	P001	2	500	ORD-10001
2024-01-02	124	P002	1	400	ORD-10002

"**Número de Pedido**" es una dimensión degenerada, ya que no requiere una tabla de dimensión separada.

Ventajas

- **Eficiencia en el almacenamiento:** Evita crear dimensiones para atributos que no tienen más detalles.
- **Facilita el análisis a nivel de transacción:** Permite identificar y filtrar transacciones específicas.

Desventajas

- **Limitación en la agrupación:** No es útil para agregaciones basadas en atributos de la dimensión.
- **Flexibilidad reducida:** Si en el futuro se necesitan más detalles, puede requerir cambios en el modelo.

4.6.6 Dimensiones basura

Las dimensiones basura son dimensiones que agrupan múltiples atributos pequeños y heterogéneos en una sola tabla de dimensión. Estos atributos, por sí solos, no justifican una dimensión independiente, pero al combinarlos, se optimiza el modelo y se simplifica el análisis.

¿Qué son las dimensiones basura?

Son dimensiones que contienen varios atributos de bajo valor analítico, como banderas (sí/no), códigos de estado o categorías pequeñas. El término "basura" no implica que los datos no sean útiles, sino que son atributos dispersos que se combinan para evitar la proliferación de dimensiones.

Ejemplo

En una empresa de comercio electrónico, podrías tener los siguientes atributos:

- **Método de pago:** Efectivo, Tarjeta, PayPal.
- **Estado del pedido:** Nuevo, Procesado, Cancelado.
- **Cliente recurrente:** Sí/No.
- **Método de envío:** Estándar, Exprés.

Estos atributos se pueden agrupar en una dimensión basura:

Dimensión_Basura_PK	Método de Pago	Estado del Pedido	Cliente Recurrente	Método de Envío
1	Efectivo	Nuevo	Sí	Estándar
2	Tarjeta	Procesado	No	Exprés

Ventajas

- **Optimización del modelo:** Reduce el número de dimensiones pequeñas y simplifica el diseño.
- **Eficiencia en el almacenamiento:** Evita múltiples tablas de dimensiones pequeñas.
- **Facilita las consultas:** Al tener atributos relacionados en una sola dimensión, las consultas son más simples.

Desafíos

- **Interpretación:** Los usuarios deben entender que la dimensión agrupa atributos heterogéneos.
- **Número de combinaciones:** Si los atributos tienen muchas combinaciones posibles, la dimensión puede crecer rápidamente.

4.6.7 Dimensiones role-playing

Las dimensiones role-playing, o dimensiones de rol, permiten reutilizar una misma tabla de dimensión en múltiples contextos dentro de una tabla de hechos. Esto es especialmente útil cuando un atributo, como la fecha, necesita ser interpretado de diferentes maneras según el escenario.

¿Qué son las dimensiones role-playing?

Son dimensiones que, aunque provienen de la misma tabla, desempeñan diferentes roles en una tabla de hechos. Por ejemplo, la dimensión de fecha puede usarse como fecha de pedido, fecha de envío y fecha de entrega dentro de la misma tabla de hechos de ventas.

Ejemplo

En una tabla de hechos de pedidos:

Pedido_ID	Fecha Pedido	Fecha Envío	Fecha Entrega	Cliente_PK	Producto_PK	Total Ventas
1001	2024-01-01	2024-01-02	2024-01-05	123	P001	500
1002	2024-01-03	2024-01-04	2024-01-07	124	P002	600

Aquí, la dimensión de fecha se utiliza tres veces con diferentes roles, pero todas las fechas provienen de la misma tabla de dimensión.

Ventajas

- **Consistencia de los datos:** Al usar la misma tabla de dimensión, se asegura la uniformidad en los atributos.
- **Simplificación del modelo:** Evita la duplicación de dimensiones para cada rol.

Desafíos

- **Complejidad en las consultas:** Es necesario renombrar las instancias de la dimensión para reflejar su rol, lo que puede complicar las consultas.
- **Posible confusión:** Los usuarios deben comprender claramente el rol que desempeña la dimensión en cada contexto.

4.6.8 Dimensiones de cambio lento

En el ámbito de los almacenes de datos y la inteligencia de negocios, es fundamental manejar adecuadamente cómo los datos dimensionales cambian con el tiempo. A este proceso se le conoce como **Slowly Changing Dimensions (SCD)** o **Dimensiones de Cambio Lento**. Existen varios tipos de SCD que se utilizan para gestionar estos cambios, cada uno con metodologías y propósitos específicos.

Tipo 0: Conservación de datos originales

En el **Tipo 0**, los datos dimensionales permanecen inalterados una vez que se han establecido. Esto significa que, independientemente de cualquier actualización o cambio en los atributos, los valores originales se conservan tal cual. Los hechos o transacciones se asocian siempre con el valor inicial del atributo. Este enfoque es útil cuando se requiere una referencia constante y los cambios posteriores no son relevantes para el análisis.

Tipo 1: Sobreescritura de datos

El **Tipo 1** implica que, cuando un atributo cambia, el valor antiguo es reemplazado directamente por el nuevo. No se mantiene ningún registro del valor anterior; simplemente se sobrescribe. Los hechos se asocian con el valor más reciente del atributo. Este método es sencillo y eficiente en términos de almacenamiento, pero tiene la desventaja de que no permite rastrear el historial de cambios, lo que puede ser problemático si se necesita análisis histórico.

ID	Name	Range
SA-23	Samsung Galaxy S7 Edge	High
IP-08	Iphone 8	High
XI-M5	Xiaomi MI 5	Medium



ID	Name	Range
SA-23	Samsung Galaxy S7 Edge	Medium
IP-08	Iphone 8	High
XI-M5	Xiaomi MI 5	Medium

Figura 21. SCD tipo 1

Fuente: bestinbi.es

Tipo 2: Registro histórico completo

El **Tipo 2** aborda la necesidad de mantener un historial detallado de todos los cambios en los atributos. Cada vez que ocurre una modificación, se añade una nueva fila a la tabla dimensional con el nuevo valor del atributo. Para gestionar estas múltiples versiones, se utilizan campos adicionales como fechas de inicio y fin de vigencia, y se asigna una clave subrogada única a cada registro. Los hechos se asocian con la versión específica del atributo que estaba activa en el momento en que ocurrió el hecho. Este enfoque permite un análisis histórico completo, pero puede aumentar significativamente el tamaño de la tabla dimensional y requiere una gestión cuidadosa de las claves y los periodos de vigencia.

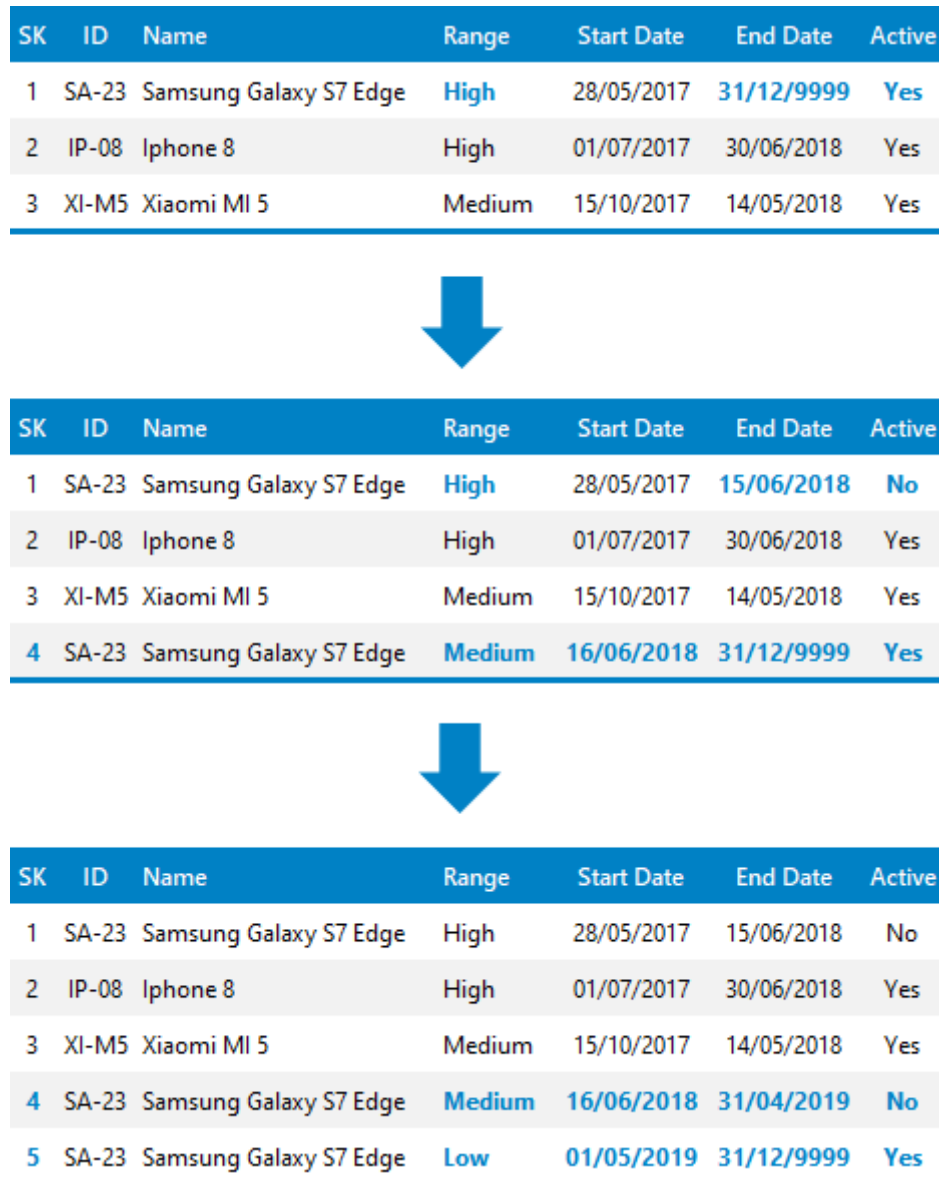


Figura 22. SCD tipo 2

Fuente: bestinbi.es

Tipo 3: Rastreo de cambios limitados

El **Tipo 3** se enfoca en almacenar únicamente el valor actual y el valor inmediatamente anterior de un atributo. Para lograr esto, se añaden columnas adicionales a la tabla dimensional para registrar el valor antiguo. Cuando se produce un cambio, el valor actual se mueve a la columna de valor anterior, y el nuevo valor se almacena como el actual. Los hechos están asociados tanto con el valor presente como con el pasado inmediato del

atributo. Este método es útil cuando solo es necesario analizar cambios recientes, pero está limitado porque no conserva un historial más allá del estado anterior.

ID	Name	Range	Previous Range
SA-23	Samsung Galaxy S7 Edge	High	
IP-08	Iphone 8	High	
XI-M5	Xiaomi MI 5	Medium	



ID	Name	Range	Previous Range
SA-23	Samsung Galaxy S7 Edge	Medium	High
IP-08	Iphone 8	High	
XI-M5	Xiaomi MI 5	Medium	

Figura 23. SCD tipo 3

Fuente: bestinbi.es

Tipo 4: Uso de mini-dimensiones

El **Tipo 4** introduce el concepto de **mini-dimensiones**. En este enfoque, se mantiene una tabla dimensional histórica completa que registra todos los cambios, y se crea una tabla adicional más pequeña que contiene solo los atributos más volátiles o de interés frecuente en su versión actual. La tabla de hechos se enlaza tanto con la dimensión histórica como con la mini-dimensión. Esto permite consultas más eficientes cuando solo se necesitan los datos actuales, al mismo tiempo que se conserva un historial completo para análisis detallados.

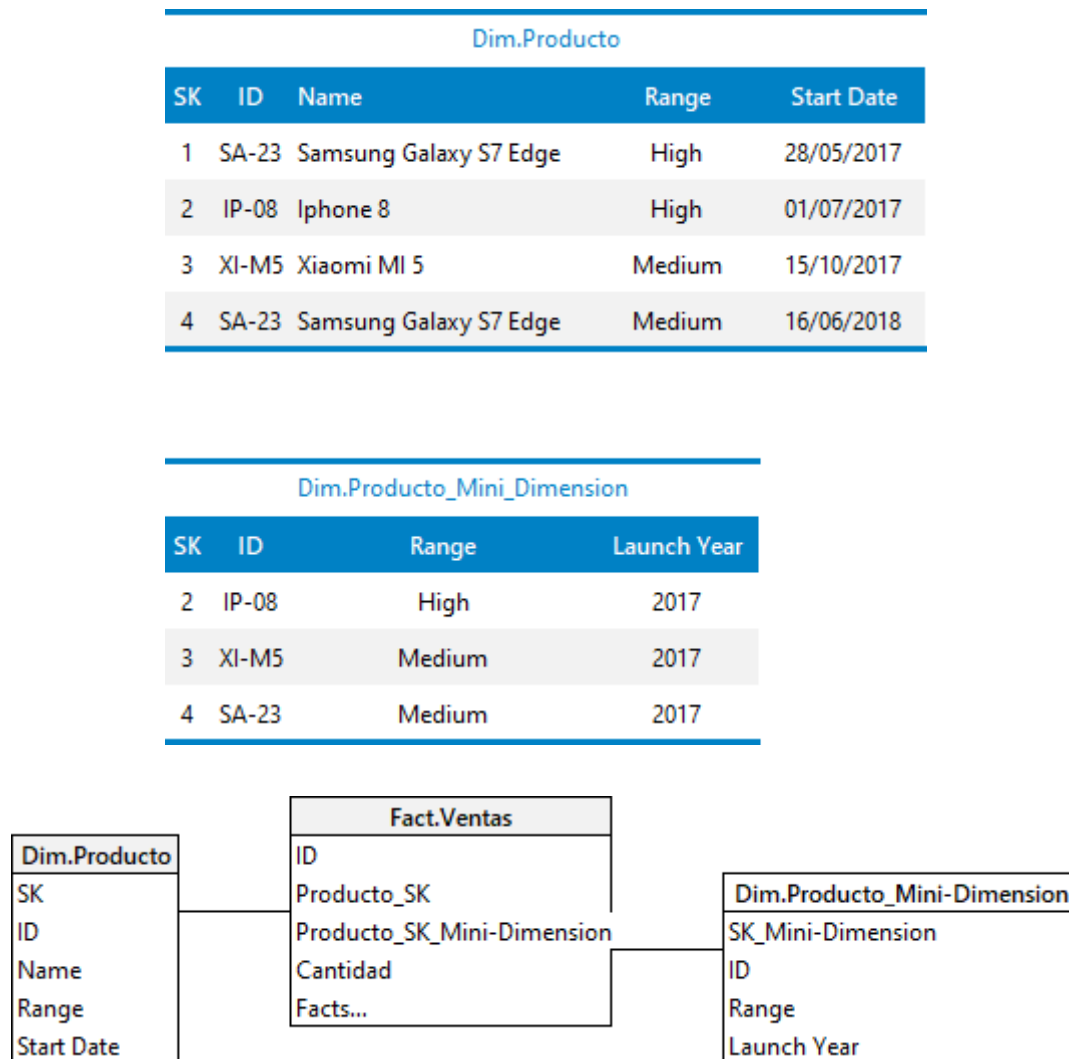


Figura 24. SCD tipo 4

Fuente: bestinbi.es

Tipo 5: Extensión del Tipo 4

El **Tipo 5** es una extensión del Tipo 4 y busca simplificar las consultas que requieren datos históricos. En este método, se añade una clave de la mini-dimensión directamente en la tabla dimensional principal. Esto permite acceder a la información histórica sin tener que unirse a la tabla de hechos, facilitando y acelerando las consultas que requieren rastrear cambios a lo largo del tiempo.

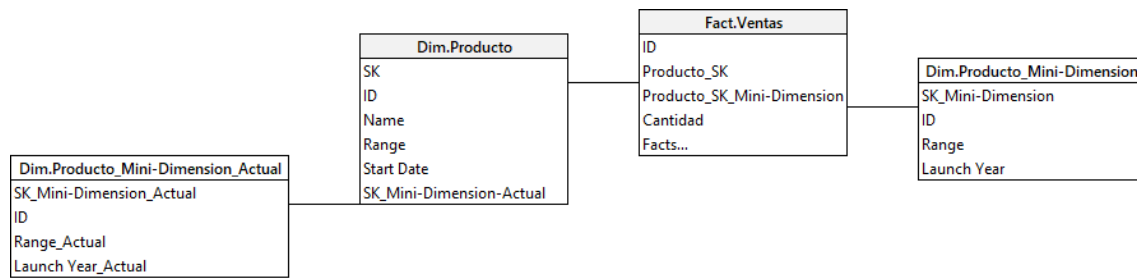


Figura 25. SCD tipo 5

Fuente: bestinbi.es

Tipo 6: Integración de Tipos 1, 2 y 3

El **Tipo 6** combina características de los Tipos 1, 2 y 3 para ofrecer una solución más completa. Se mantiene un historial completo de cambios mediante la creación de nuevas filas (Tipo 2), se sobrescribe el valor actual en la tabla dimensional (Tipo 1), y se conserva el valor anterior en una columna separada (Tipo 3). Esto permite analizar los datos desde múltiples perspectivas: el estado actual, el estado en el momento del hecho y el estado anterior. Aunque es más complejo de implementar, ofrece una flexibilidad y profundidad de análisis superiores.

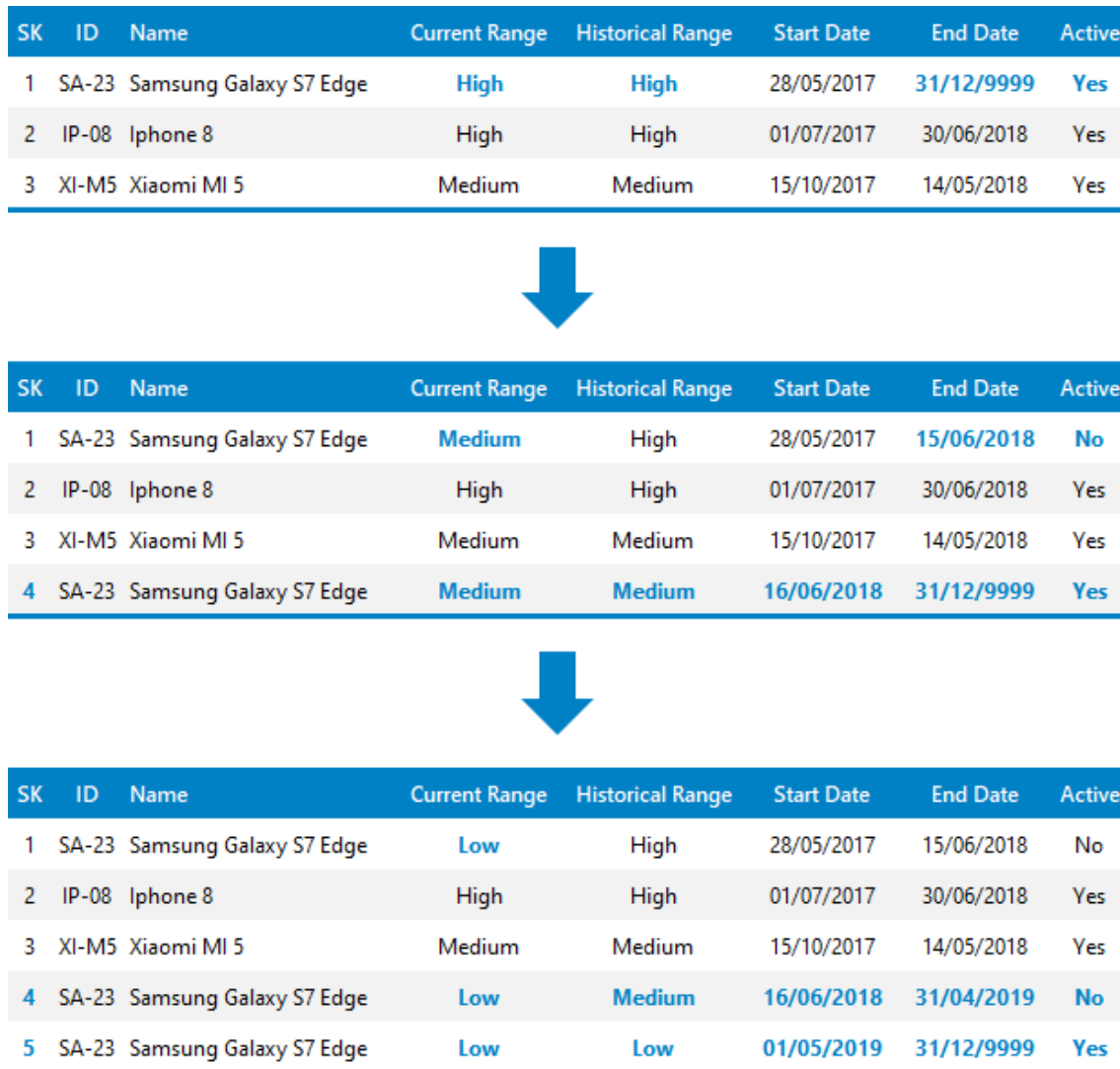


Figura 26. SCD tipo 6

Fuente: bestinbi.es

Tipo 7: Tabla adicional para datos actuales

El **Tipo 7** amplía el enfoque del Tipo 6 añadiendo una tabla separada que contiene únicamente la versión más reciente de los datos. Esta tabla se utiliza para mejorar el rendimiento en consultas y reportes que solo requieren información actual. Los hechos se asocian tanto con el valor del atributo en el momento en que ocurrió el hecho como con el valor actual, proporcionando una visión completa y eficiente de los datos históricos y presentes.

SK	ID	Name	Current Range	Historical Range	Start Date	End Date	Active
1	SA-23	Samsung Galaxy S7 Edge	Low	High	28/05/2017	15/06/2018	No
2	IP-08	Iphone 8	High	High	01/07/2017	30/06/2018	Yes
3	XI-M5	Xiaomi MI 5	Medium	Medium	15/10/2017	14/05/2018	Yes
4	SA-23	Samsung Galaxy S7 Edge	Low	Medium	16/06/2018	31/04/2019	No
5	SA-23	Samsung Galaxy S7 Edge	Low	Low	01/05/2019	31/12/9999	Yes

SK	ID	Name	Current Range	Launch Year
2	IP-08	Iphone 8	High	2017
3	XI-M5	Xiaomi MI 5	Medium	2017
5	SA-23	Samsung Galaxy S7 Edge	Low	2017

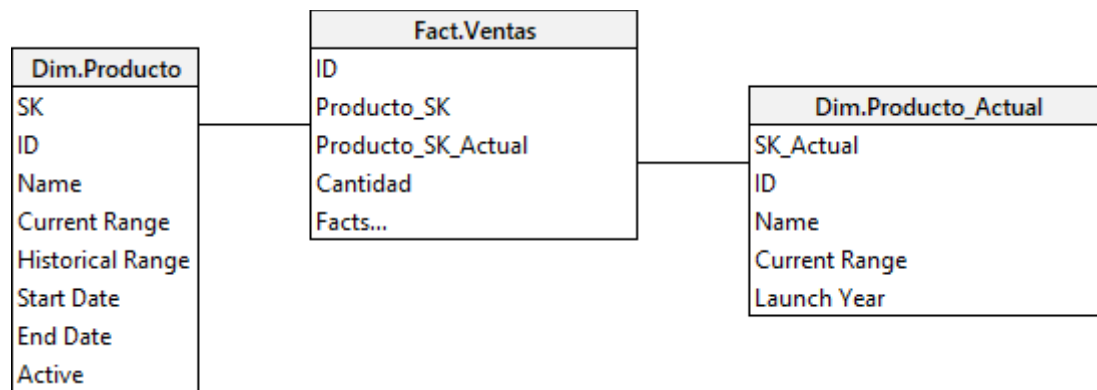


Figura 27. SCD tipo 7

Fuente: bestinbi.es

4.7 Desarrollando el diseño dimensional

El diseño dimensional es un proceso fundamental para construir un almacén de datos que responda eficazmente a las necesidades de análisis de una organización. Este enfoque se centra en estructurar los datos de manera que sean fácilmente accesibles y comprensibles para los usuarios de negocio. A continuación, exploraremos los pasos esenciales para desarrollar un diseño dimensional, ilustrándolos con ejemplos prácticos.

1. Identificación de los procesos del negocio

El primer paso es identificar claramente los procesos del negocio que se van a modelar. Un proceso de negocio es una actividad o conjunto de actividades que una organización realiza para generar valor o cumplir sus objetivos. Al seleccionar qué procesos incluir, definimos el alcance del modelo y establecemos las bases para el diseño de las tablas de hechos y dimensiones.

Por ejemplo, si estamos diseñando un almacén de datos para una empresa de comercio electrónico, podríamos enfocarnos en los siguientes procesos clave:

- **Ventas de productos:** Registrar cada transacción de venta, capturando detalles como el producto vendido, el cliente, la fecha y el monto de la venta.
- **Gestión de inventarios:** Controlar las existencias y movimientos de productos en el almacén, incluyendo entradas, salidas y niveles de stock.
- **Devoluciones:** Hacer seguimiento de los productos devueltos por los clientes, incluyendo las razones de devolución y el estado del reembolso o reemplazo.

Al identificar estos procesos, nos aseguramos de que el almacén de datos refleje las actividades críticas de la empresa y proporcione información relevante para la toma de decisiones.

2. Definición del nivel de detalle o granularidad

Una vez identificados los procesos, es crucial definir el nivel de detalle o granularidad del modelo. La granularidad se refiere a la unidad más pequeña de datos que se almacenará en la tabla de hechos. Esta decisión afecta tanto la cantidad de datos almacenados como la capacidad de análisis.

Continuando con el ejemplo de ventas de productos, podemos considerar dos niveles de granularidad:

- **Granularidad transaccional:** Cada fila en la tabla de hechos representa una transacción individual. Por ejemplo, una compra realizada por un cliente específico en una fecha y hora determinadas. Este nivel permite un análisis detallado de cada transacción, como patrones de compra por cliente o análisis de ventas por hora del día.
- **Granularidad diaria:** Cada fila representa el total de ventas de un producto en un día. Esto reduce el tamaño de la tabla y simplifica el almacenamiento, pero limita la capacidad para analizar transacciones individuales. Es útil para identificar tendencias diarias y comparar el desempeño entre diferentes días.

La elección del nivel de granularidad debe alinearse con las necesidades de análisis del negocio. Si se requiere un análisis detallado del comportamiento del cliente, la granularidad transaccional es más apropiada. Si el enfoque está en tendencias generales, la granularidad diaria puede ser suficiente.

3. Identificación de las dimensiones

Las dimensiones son tablas que contienen atributos descriptivos relacionados con los hechos y proporcionan contexto para el análisis. En este paso, identificamos las entidades que describen el entorno en el que ocurren los hechos, respondiendo a preguntas como quién, qué, cuándo, dónde y cómo.

Para el proceso de ventas de productos, las dimensiones podrían ser:

- **Dimensión de Producto:** Incluye información sobre los productos vendidos, como nombre, categoría, precio y proveedor. Esto permite analizar las ventas por tipo de producto, categoría o proveedor.
- **Dimensión de Cliente:** Contiene datos sobre los clientes, incluyendo nombre, ubicación, edad y segmento de mercado. Con esta dimensión, podemos realizar análisis demográficos y segmentar las ventas por grupos de clientes.

- **Dimensión de Fecha:** Proporciona la estructura temporal para el análisis, permitiendo examinar las ventas por día, mes, trimestre o año. También puede incluir indicadores como día de la semana, festivos o temporadas.
- **Dimensión de Tienda** (si aplica): Describe las tiendas físicas donde se realizan las ventas, con detalles como ubicación, tamaño y gerente. Esto es útil para comparar el desempeño entre diferentes tiendas o regiones.

Al identificar estas dimensiones, creamos un marco que facilita el análisis multidimensional y permite a los usuarios explorar los datos desde diversas perspectivas.

4. Identificación de los hechos

Los hechos son los datos numéricos y medibles que representan los eventos o transacciones del negocio. Son el núcleo del análisis cuantitativo y suelen incluir métricas clave como ventas, cantidades y costos.

En una tabla de hechos de ventas de productos, podríamos incluir:

- **Cantidad vendida:** El número de unidades vendidas en cada transacción o período.
- **Precio de venta:** El precio por unidad del producto en la transacción. Esto ayuda a analizar estrategias de precios y descuentos.
- **Total de ventas:** El monto total de la venta, calculado como la cantidad vendida multiplicada por el precio de venta. Es fundamental para evaluar el rendimiento financiero.
- **Descuento aplicado:** Si se otorgó un descuento, se registra el monto o porcentaje. Esto permite analizar el impacto de promociones en las ventas.

Al identificar los hechos relevantes, establecemos las métricas que serán objeto de análisis y seguimiento en el almacén de datos.

5. Implementación de bases de datos NoSQL

En el contexto actual de Big Data, las bases de datos NoSQL (Not Only SQL) han emergido como una alternativa poderosa a las bases de datos relacionales tradicionales. Su capacidad para manejar grandes volúmenes de datos, flexibilidad en la estructura y escalabilidad horizontal las hace ideales para ciertas aplicaciones.

5.1 Introducción a NoSQL

Las bases de datos NoSQL ofrecen un enfoque diferente al almacenamiento y gestión de datos, adaptándose mejor a ciertos tipos de aplicaciones y necesidades.

5.1.1. Diferencias entre bases de datos relacionales y NoSQL

- **Estructura de los datos:** Las bases de datos relacionales utilizan un esquema fijo basado en tablas y relaciones. Requieren definir de antemano la estructura de los datos, lo que puede limitar la flexibilidad. Por el contrario, las bases de datos NoSQL no requieren un esquema rígido, permitiendo almacenar datos estructurados, semiestructurados o no estructurados. Esto facilita la adaptación a cambios en los datos sin necesidad de modificar esquemas predefinidos.
- **Escalabilidad:** Las bases de datos relacionales escalan principalmente de forma vertical (mejorando el hardware del servidor). Las bases de datos NoSQL están diseñadas para escalar horizontalmente, añadiendo más nodos al clúster para manejar mayores volúmenes de datos y aumentar el rendimiento.
- **Consistencia y disponibilidad:** Las bases de datos relacionales garantizan una alta consistencia mediante el cumplimiento de las propiedades ACID. Son ideales para aplicaciones donde la precisión es crítica. Las bases de datos NoSQL siguen el teorema CAP, permitiendo flexibilidad en la consistencia y priorizando la disponibilidad y tolerancia a fallos, lo que es beneficioso para aplicaciones distribuidas globalmente.

- **Consultas:** Las bases de datos relacionales utilizan SQL, un lenguaje estandarizado para realizar consultas complejas. Las bases de datos NoSQL no tienen un lenguaje de consulta unificado; cada tipo utiliza su propio lenguaje o API. Por ejemplo, MongoDB utiliza un lenguaje basado en JSON.

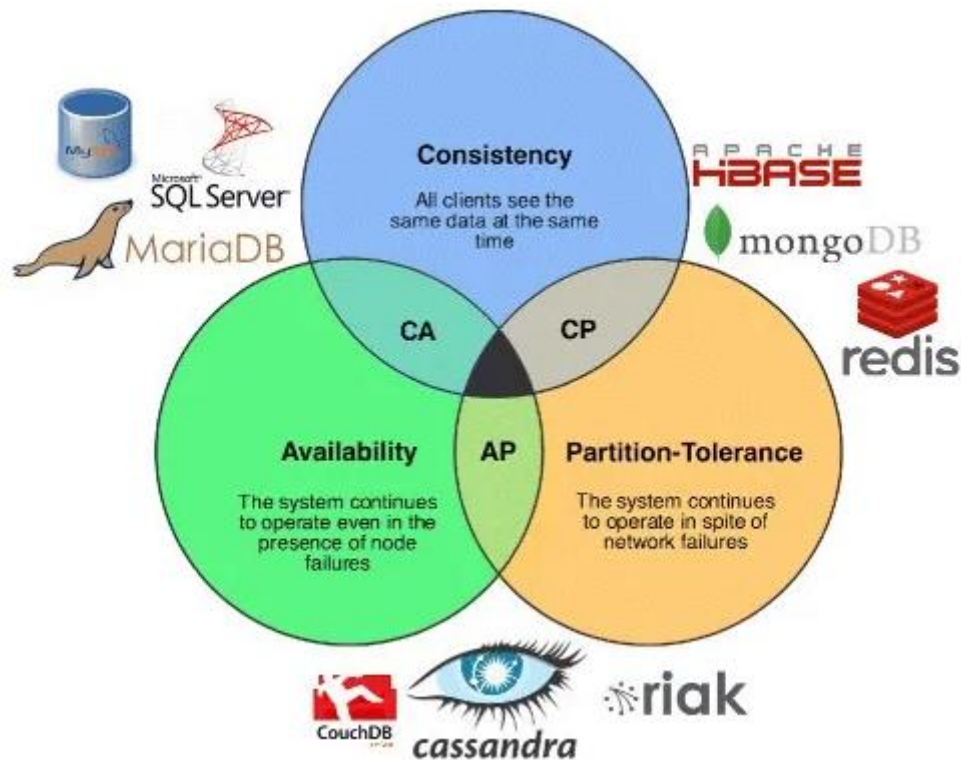


Figura 28. Teorema CAP

Fuente: medium.com

5.1.2. Modelo ACID y teorema CAP

Tras nombrarlos en el punto anterior, es esencial profundizar en el entendimiento del modelo ACID y su relevancia en el diseño y gestión de sistemas de bases de datos modernos. Las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) son pilares fundamentales que garantizan la fiabilidad y coherencia de las transacciones en los sistemas de bases de datos tradicionales.

Atomicidad garantiza que una transacción es una unidad indivisible de trabajo. Esto significa que una transacción compuesta por múltiples operaciones se ejecutará completamente o no se ejecutará en absoluto. Si alguna de las operaciones falla, todas las

operaciones previas se deshacen, asegurando que la base de datos no quede en un estado inconsistente. Por ejemplo, en una transferencia bancaria, si se debita una cuenta, pero falla el crédito en la otra, la atomicidad asegura que ambas operaciones se revertirán para mantener la integridad de los datos.

Consistencia asegura que cada transacción lleve a la base de datos de un estado válido a otro, manteniendo intactas todas las reglas y restricciones definidas, como la integridad referencial y las restricciones de unicidad. Esto significa que cualquier transacción no válida que viole las reglas del sistema será rechazada, preservando la integridad de la base de datos. Por ejemplo, si una regla establece que el saldo de una cuenta no puede ser negativo, la consistencia garantiza que ninguna transacción pueda resultar en un saldo negativo.

Aislamiento es fundamental en entornos donde múltiples transacciones ocurren simultáneamente. Garantiza que las operaciones de una transacción sean invisibles para otras hasta que se completen, evitando conflictos y resultados indeterminados. Esto previene problemas como lecturas sucias, lecturas no repetibles y fenómenos fantasma. Por ejemplo, si dos transacciones intentan actualizar el mismo registro al mismo tiempo, el aislamiento asegura que una se procese completamente antes de que la otra comience, evitando interferencias entre ellas.

Durabilidad garantiza que, una vez que una transacción ha sido confirmada, sus efectos persistirán en la base de datos, incluso ante fallos del sistema, cortes de energía u otros desastres. Esto se logra mediante técnicas como la escritura de registros de transacciones en medios de almacenamiento persistentes antes de confirmar la transacción al usuario. La durabilidad es crucial para asegurar que los datos no se pierdan y que el sistema pueda recuperarse correctamente después de un fallo.

En el contexto de Big Data y las bases de datos NoSQL, el cumplimiento estricto de las propiedades ACID puede presentar desafíos significativos. Los sistemas de Big Data a menudo requieren escalabilidad horizontal, distribuyendo datos y cargas de trabajo a través de múltiples nodos y centros de datos geográficamente dispersos. Mantener las propiedades ACID en un entorno distribuido de esta naturaleza puede introducir una complejidad considerable y afectar negativamente el rendimiento y la disponibilidad del sistema.

Aquí es donde entra en juego el Teorema CAP (Consistencia, Disponibilidad y Tolerancia a Particiones), que establece que es imposible para un sistema distribuido garantizar simultáneamente las tres propiedades. En la práctica, los sistemas deben hacer concesiones y priorizar dos de ellas a expensas de la tercera. Muchos sistemas NoSQL optan por sacrificar la consistencia inmediata en favor de la disponibilidad y la tolerancia a particiones, adoptando un modelo de consistencia eventual.

El enfoque BASE (Basically Available, Soft state, Eventual consistency) es una alternativa al modelo ACID que muchos sistemas NoSQL adoptan para lograr mayor escalabilidad y flexibilidad. Bajo este modelo, el sistema garantiza que los datos estarán eventualmente consistentes, permitiendo que diferentes nodos tengan valores temporales divergentes que se resolverán con el tiempo. Esto es aceptable en aplicaciones donde una consistencia inmediata no es crítica, como en redes sociales o sistemas de análisis en tiempo real, donde la velocidad y la disponibilidad son más importantes.

Sin embargo, esta relajación de la consistencia puede no ser adecuada para todas las aplicaciones. En sectores como el financiero, sanitario o gubernamental, donde la integridad y consistencia de los datos son vitales, las propiedades ACID siguen siendo indispensables. Por ejemplo, en transacciones bancarias, cualquier inconsistencia puede tener consecuencias graves, por lo que se requiere un cumplimiento estricto de ACID para garantizar la exactitud y confiabilidad de las operaciones.

Además, algunos sistemas NoSQL han comenzado a incorporar características que permiten cierto grado de soporte ACID en operaciones limitadas o en contextos específicos. Por ejemplo, bases de datos como MongoDB y Cassandra han introducido transacciones que permiten operaciones atómicas en documentos individuales o en conjuntos limitados de datos. Esto representa un esfuerzo por equilibrar la necesidad de consistencia con los beneficios de escalabilidad y rendimiento que ofrecen los sistemas NoSQL.

5.1.3. Cuándo utilizar bases de datos NoSQL en lugar de SQL

Las bases de datos NoSQL son especialmente adecuadas en situaciones como:

- **Grandes volúmenes de datos:** Cuando se manejan cantidades masivas de datos distribuidos, como en redes sociales o aplicaciones IoT.
- **Datos no estructurados o semiestructurados:** Si los datos varían en estructura o cambian con frecuencia, NoSQL ofrece la flexibilidad necesaria.
- **Altos requisitos de disponibilidad y baja latencia:** Para aplicaciones en tiempo real que requieren respuestas rápidas y disponibilidad constante.
- **Cambios rápidos en la estructura de datos:** Cuando la estructura de los datos puede cambiar rápidamente, NoSQL permite agregar o eliminar campos sin afectar la operación general.
- **Aplicaciones distribuidas geográficamente:** Bases de datos como Cassandra o DynamoDB están diseñadas para manejar datos repartidos en varios centros de datos a nivel mundial.

5.2 Tipos de bases de datos NoSQL

Las bases de datos NoSQL se clasifican en diferentes tipos según su modelo de datos, cada uno optimizado para casos de uso específicos.

5.2.1. Bases de datos documentales

Estas bases de datos almacenan datos en forma de documentos, generalmente en formato JSON o BSON. Cada documento puede tener una estructura diferente, permitiendo una gran flexibilidad. Ejemplos incluyen MongoDB y CouchDB.

- **Caso de uso:** Ideal para aplicaciones que manejan datos semiestructurados, como catálogos de productos, donde cada producto puede tener diferentes atributos.

5.2.2. Bases de datos clave-valor

Almacenan datos como pares de clave-valor, siendo la clave un identificador único y el valor cualquier tipo de dato. Ejemplos son Redis y DynamoDB.

- **Caso de uso:** Excelente para accesos rápidos a datos, como en sistemas de caché, gestión de sesiones de usuario o colas de mensajes.

5.2.3. Bases de datos columnar

Organizan los datos por columnas en lugar de filas, lo que mejora la eficiencia en consultas sobre grandes volúmenes de datos. Ejemplos incluyen Cassandra y HBase.

- **Caso de uso:** Adecuadas para almacenamiento distribuido de datos masivos, como registros de logs o información de sensores en IoT.

5.2.4. Bases de datos de grafos

Almacenan datos en forma de nodos y relaciones (aristas), permitiendo modelar y consultar relaciones complejas. Ejemplos son Neo4j y ArangoDB.

- **Caso de uso:** Ideales para aplicaciones que requieren analizar redes o relaciones, como redes sociales, sistemas de recomendación y detección de fraudes.

5.3 Modelado en NoSQL

El modelado de datos en bases NoSQL difiere del enfoque relacional. Se basa en cómo se accede y se consulta la información, priorizando el rendimiento y la escalabilidad.

5.3.1. Enfoques de modelado de datos en NoSQL

- **Desnormalización:** En NoSQL, es común desnormalizar los datos, es decir, duplicarlos para mejorar el rendimiento de las consultas. Esto elimina la necesidad de realizar uniones complejas.

Ejemplo: En MongoDB, podríamos almacenar la información del proveedor dentro del documento del producto, en lugar de tener colecciones separadas.

- **Modelado basado en consultas:** El diseño del esquema se basa en las consultas más frecuentes que realizará la aplicación.

Ejemplo: Si necesitamos acceder a los pedidos de un cliente por fecha en Cassandra, creamos una tabla con la clave primaria compuesta por cliente_id y fecha_pedido, optimizando la recuperación de datos.

5.3.2. Ejemplos prácticos

- **MongoDB:** Para un catálogo de productos, cada documento representa un producto con sus atributos, permitiendo almacenar variaciones y detalles específicos.
- **Redis:** Usado como caché para almacenar sesiones de usuario, donde la clave es el ID de sesión y el valor contiene los datos asociados.
- **Cassandra:** En una red social, almacenar publicaciones de usuarios distribuidas en múltiples nodos, permitiendo consultas eficientes por usuario y fecha.
- **Neo4j:** Modela usuarios y sus relaciones en una red social, facilitando el análisis de conexiones y recomendaciones.

5.4 Integración con almacenes de datos y lagos de datos

Las bases de datos NoSQL pueden integrarse con almacenes de datos y lagos de datos para aprovechar sus respectivas fortalezas.

- **Coexistencia con almacenes de datos:** Las bases NoSQL manejan operaciones en tiempo real y datos no estructurados, mientras que los almacenes de datos permiten análisis profundos de datos estructurados.

Ejemplo: Datos transaccionales almacenados en Cassandra se exportan a un almacén de datos como Amazon Redshift para análisis de comportamiento del cliente.

- **Integración con lagos de datos:** Los datos de NoSQL pueden ser fuente para lagos de datos, almacenando grandes volúmenes en su formato original para análisis posteriores.

Ejemplo: Datos de MongoDB se exportan a un lago de datos en Amazon S3 para ser procesados mediante Machine Learning.

5.5 Gestión del rendimiento y escalabilidad

Diseñar una base de datos NoSQL eficiente requiere considerar aspectos clave de rendimiento y escalabilidad.

5.5.1. Diseño para grandes volúmenes de datos

- **Escalabilidad horizontal:** Añadir más nodos al clúster para distribuir datos y carga de trabajo.

Ejemplo: En Cassandra, los datos se distribuyen automáticamente entre nodos, permitiendo manejar grandes cantidades de datos eficientemente.

- **Optimización de esquemas:** Diseñar esquemas que minimicen consultas complejas y se adapten a las consultas más frecuentes.

Ejemplo: En DynamoDB, usar claves compuestas para mejorar el rendimiento de consultas por múltiples dimensiones.

5.5.2. Técnicas de replicación, particionamiento y consistencia

- **Replicación:** Copiar datos en múltiples nodos para mejorar la disponibilidad.

Ejemplo: MongoDB utiliza replica sets para asegurar que los datos estén disponibles incluso si un nodo falla.

- **Particionamiento (Sharding):** Dividir datos entre nodos para mejorar escalabilidad y rendimiento.

Ejemplo: Cassandra particiona datos usando una función de hash en la clave de partición.

- **Consistencia:** Ajustar el nivel de consistencia según las necesidades, equilibrando disponibilidad y precisión.

Ejemplo: En Cassandra, se puede configurar la consistencia eventual para mejorar la disponibilidad en aplicaciones donde la precisión inmediata no es crítica.

Bibliografía

1. R. Kimball y M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3ª ed. Wiley, 2013.
2. W. H. Inmon, *Building the Data Warehouse*, 4ª ed. Wiley, 2005.
3. T. Erl, R. Khattak y P. Buhler, *Big Data Fundamentals: Concepts, Drivers & Techniques*, 1ª ed. Prentice Hall, 2016.
4. P. Zikopoulos, C. Eaton, D. Deroos, T. Deutsch y G. Lapis, *Understanding Big Data: Analytics for Enterprise Class Hadoop and Streaming Data*, 1ª ed. McGraw-Hill, 2011.
5. J. G. Grimes, *Cloud Computing: Theory and Practice*, 1ª ed. Morgan Kaufmann, 2012.
6. S. W. Ambler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*, 1ª ed. Addison-Wesley, 2012.
7. D. G. Linoff y G. S. Berry, *Data Mining Techniques: For Marketing, Sales, and Customer Relationship Management*, 3ª ed. Wiley, 2011.
8. Thusoo, R. Murthy, J. S. Sarma, Z. Shao, N. Jain, H. Liu, y A. Seth, "Hive - A Petabyte Scale Data Warehouse Using Hadoop," in *Proceedings of the 26th International Conference on Data Engineering*, 2010, pp. 996-1005.
9. J. Dean y S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107-113, Jan. 2008.
10. M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 1ª ed. O'Reilly Media, 2017.